

1. ATALA

Lengoiak eta Ingeniaritza

EGILEA

Aitor Aretxaga Agirre

ZENTROA

Uni Eibar-Ermua

EDIZIOA

v1.1 (Laneki — FP Euskadi materialean oinarritua)

IKASTURTEA

2026 — 2027

Lengoaiak eta Ingeniaritza

Sarrera

Imajina ezazu zure lehen eguna lan-postu berri batean. Sartu zara enpresa-bulegoetara, ordenagailu berria daukazu mahai gainean, eta zure tutoreak esan dizu: "Atzo Mariak Python script bat utzi zigun salmenta-datuak prozesatzeko. Exekuta ezazu, eta gero atal txiki bat aldatu behar dugu". Ordenagailua piztu, kodea ireki, eta... Python ez dago instalatuta. Python instalatzen duzu — bertsio okerra. Beste bertsio bat probatzen duzu — orain liburutegi bat falta da. `pip install`-ekin liburutegia jartzen duzu — bertsio-gatazka bat dago beste liburutegi batekin. Lehen ordua ia pasa egin da eta oraindik ez duzu kodea exekutatu.

Eszena hau egunero gertatzen da garatzaile berrientzat eta baita esperientziadunentzat ere. Software profesionalaren oinarrian ez dago lengoiaia bat soilik, baizik eta **ingurune profesional** oso bat: editorea, ingurune birtualak, pakete-kudeaketa, bertsio-kontrola eta lan-fluxu egokiak. Hori prest dagoenean, kodearen edukian zentratu ahal da; baina hori prest ez dagoenean, ezerk ez du funtzionatzen.

Gai honek hori prestatzen irakatsiko dizu. Ez da kontu burokratikoa: enpresa-mailan, asteko ordu asko aurrezten dira ingurune ondo konfiguratutako batean. Eta zure ikaskuntza bera ere askoz azkarragoa izango da, kontzeptu berriek hartuko baitute zure denbora, ez oztopo logistikoek.

! Ideia nagusia *Ingurune profesional sendoa ez da edukirako lagungarria, kodearen edukira heltzeko bidea da. Enpresako errealitatearekin lotura mehea bada, garatuko diren kontzeptuek ezertarako gutxitarako balioko dute, ezingo baitira praktikan jarri.*

1. Programa informatiko baten egitura eta AArako lengoaiak

Atal honetan programazioaren oinarritzko kontzeptuetatik abiatuko gara: zer da programa informatiko bat, zein etapa ditu, eta zein dira AAre garapenean gehien erabiltzen diren programazio-lengoaiak. BOEko Errege Dekretua (RD 279/2021) lehen ikaskuntza-emaizaren oinarria da: lengoaiak karakterizatu eta haien egokitasuna baloratu. Atal hau ez da formala edo teorikoa soilik, egunero erabakiak hartzen dira sektorean: zein lengoaia aukeratu proiektu honetarako? Zein liburutegi instalatu? Hori egiten ikasiko duzu.

1.1 Programa informatiko baten egitura

Programa informatiko bat atazak gauzatzeko prozesatzaileari aginduak ematen dizkion instrukzio-multzo bat da. Ez da magia, eskailera bat bezalakoa da, ondo definitutako urratsekin: makinak aginduak banan-banan exekutatzen ditu, akatsik gabe (akatsa gizakiak idatzi badu bakarrik gertatzen da).

GARAPEN-ZIKLOA: SEI ETAPAK

Software baten garapenak normalean sei etapa ditu, askotan ziklikoki errepikatzen direnak. Enpresetan etapek izen ezberdinak izan ditzakete (metodologiaren arabera, [agile](#) edo [waterfall](#) izan), baina mamia berdina da:

1. **Analisia eta diseinua** — arazoa ulertu, behar funtzionalak zehaztu, datu-fluxua eta algoritmoak diseinatu. Hemen sortzen dira diagramak (UML, BPMN) eta dokumentazioa. Esate baterako: bezeroaren eskakizuna entzun, "salmentak ikusteko aplikazio bat behar dut" → diseinatu zer pantaila izango dituen, zein datu erabiliko dituen, eta abar.
2. **Idazketa (kodifikazioa)** — editorean (Antigravity, VS Code) kodea idatzi. Hau da atal honetan gehien landuko duguna.
3. **Konpilazioa edo interpretazioa** — programa exekutagarri bihurtu. Python interpretatua dago (ez da konpilatu behar aurretik); Java edo C++-en, ostera, konpilatu behar da exekutatu aurretik.
4. **Testing** — programa probatu (unit testak, integrazio-testak, sistema-testak). Programaren akatsak topatu erabiltzaileek aurkitu baino lehen.
5. **Hedapena (deployment)** — programa erabiltzaileek erabili dezaten ipini (hodeian, mugikorrean, edo mahaigaineko aplikazio gisa).
6. **Mantentzea** — akatsak konpondu, ezaugarri berriak gehitu, errendimendua hobetu, segurtasuna eguneratu. Software baten bizitzaren % 60-80 etapa honetan biltzen da askotan.

Software-garapenaren bizi-zikloa



1. Irudia: Software-garapenaren sei etapen zikloa, etapa bakoitzaren deskribapen laburrarekin.

PROGRAMA BATEN ANATOMIA

Edozein programa atal hauek osatzen dute funtsean, lengoaia edozein delarik ere:

- **Datuak** — aldagaiak, konstanteak, datu-egiturak (zerrendak, hiztegiak, objektuak).
- **Kontrol-fluxua** — erabakiak (if/else), errepikapenak (for/while), salbuespenak.
- **Funtzioak eta moduluak** — berrerabilgarriak diren kode-unitate logikoak.
- **Inputs/Outputs** — erabiltzailearen sarrera, fitxategiak, sareko deiak.
- **Errore-kudeaketa** — ustekabeko egoerei aurre egin (try/except).

```

# Programa baten egiturazko adibidea (Python)

# 1. Inportazioak — kanpoko paketeen kargaketa
import json
from pathlib import Path

# 2. Konstanteak eta konfigurazioa
MAX_AHALEGINAK = 3
  
```

```
CONFIG_PATH = Path("config.json")

# 3. Funtzioak - logika berrerabilgarria
def kargatu_konfigurazioa(path: Path) -> dict:
    """Konfigurazio-fitxategia kargatu eta itzuli."""
    try:
        with open(path, "r") as f:
            return json.load(f)
    except FileNotFoundError:
        return {"lehenetsia": True}

# 4. Programa nagusia
def main():
    config = kargatu_konfigurazioa(CONFIG_PATH)

    for i in range(MAX_AHALEGINAK):
        izena = input("Sartu zure izena: ")
        if izena.strip():
            print(f"Kaixo, {izena}!")
            break
    else:
        print("Errorea: izen baliokun bat behar duzu.")

# 5. Sarrera-puntua - script gisa exekutatzean
if __name__ == "__main__":
    main()
```

Q Adibidea: Tutoreak zer eskatuko dizun lehen astean Imaginatu Mariak prozesatzen duen salmenta-script bat. CSV bat irakurri, salmenten guztizko zenbatekoa kalkulatu eta JSON gisa esportatu egiten du. Hori ondo idazteko, **datuak** (CSV-tik datozenak), **kontrol-fluxua** (errenkadak prozesatu), **funtzio bat** (zenbatekoa kalkulatu) eta **errore-kudeaketa** (zer egin CSV-a ez badago?) behar dira. Programa horrek 30 lerro ditu, baina anatomia bera dauka 30.000 lerro dituen aplikazio batek. Egitura ikasten baduzu, edozein tamainara aplikatzen da.

Δ Kontuz Hasiberri askok zuzenean idazten dute kodea, planik gabe ("Lehenik kode-pixka bat probatuko dut"). Hori onargarria da 10 lerroko script batean; baina 100 lerro baino gehiago dituen programa batean **planifikatzea** ahazteak erabateko nahasmena ekartzen du. Beti hartu 10 minutu paperean ideia idazteko exekutatzen hasi baino lehen.

□ Ariketa 1.1 · 15 min · banaka Egoera: Eskolako programatze-laguntzailea zara. Beste ikasle batek script bat egin du Python-en, baina lerro guztiak nahastuta daude (inportazioak erdian, konstanteak amaieran, eta abar). **Eginkizuna:** Idatzi paperean script soil bat 6 ataletan banatuta (1-5 puntuak + sarrera-puntua), erabiltzaileari NIF-a galdetzen diona eta hori ondo formatuta dagoen baieztatzen duena (8 zenbaki + letra bat). **Eztabaidatzeko:** Zer gertatuko litzateke `if __name__ == "__main__":` lerroa ez bagenu sartuko? Noiz exekutatuko litzateke `main()` funtzioa?

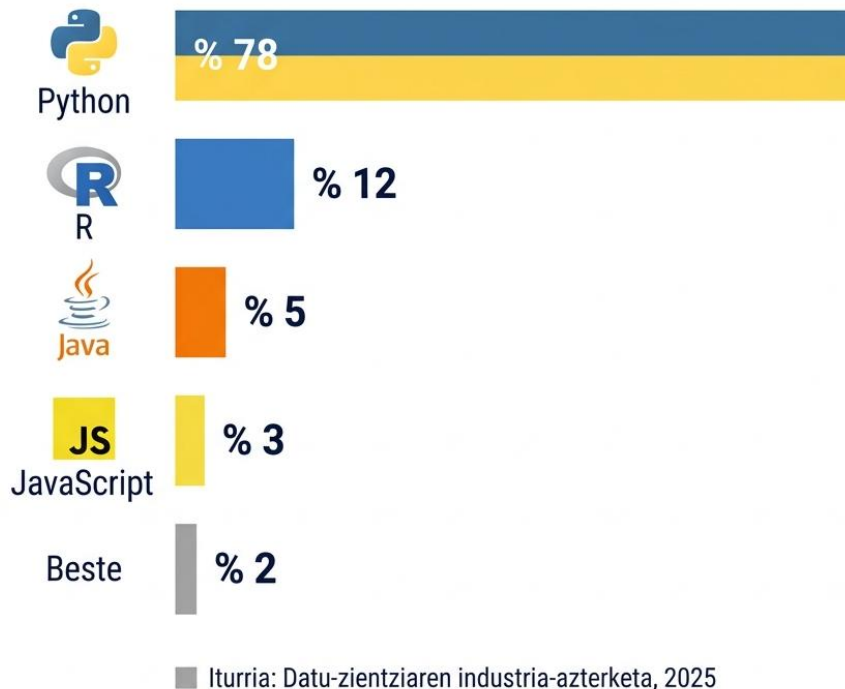
1.2 AArako programazio-lengoaiei alderaketa

AArako lengoaia asko daude merkatuan. BOEko Errege Dekretuak hauek aipatzen ditu zuzenean: **Python, R, Java, JavaScript, NodeJS, JSON eta beste batzuk**. Bakoitzaren ezaugarriak eta erabilera tipikoa ezagutzea funtsezkoa da edozein AA-proiektutan **lengoaia egokia aukeratzeko**. Garrantzitsua da: aukeraketa okerra eginez gero, hilabeteak galdu ditzakezu lengoaia desegokia konpontzen.

KONPARAKETA-TAULA

Lengoaia	Mota	Indargunea AAn	Erabilera tipikoa	Adibide-erabiltzaileak
Python	Interpretatua, dinamikoa	ML ekosistema zabalena	Datu-zientzia, ML, GenAI	OpenAI, Google, Meta
R	Interpretatua, dinamikoa	Estatistika eta bistaratzea	Datu-azterketa akademikoa, biostatistika	Roche, Genentech, unibertsitateak
Java	Konpilatua, estatikoa	Errendimendu altua, ekoizpen-sistema sendoak	Big Data (Hadoop, Spark), enpresa-aplikazioak	LinkedIn, Twitter, banketxeak
JavaScript	Interpretatua, dinamikoa	Nabigatzaileen integrazioa	TensorFlow.js, AA web-aplikazioetan	Web-aplikazioak orokorrean
NodeJS	JS server-aldean	API-zerbitzariak, denboran errealean	LLM API-en integrazioa backenden	Backend-ak Gemini/OpenAI-rekin

IArako lengoaien adopzioa industrian (2025)



2. Irudia: AArako lengoaia nagusien adopzio-tasa industria-azterketen arabera, Python nagusi gisa eta R akademian.

ZERGATIK DA PYTHON NAGUSI

Python da gaur egun AAre *de facto* lengoaia. Arrazoiak:

- Sintaxia sinplea eta irakurterraza** — edozein hizkuntzatatik datorren garatzaileak egun gutxitan ulertzen du.
- Liburutegi-ekosistema bikaina** — NumPy, Pandas, scikit-learn, PyTorch, TensorFlow, Hugging Face, LangChain — denak Python-en idatziak edo Python interfaze nagusiarekin.
- Komunitate handia** — AAko [ikerketa-paper](#) gehienek (% 80 baino gehiago) Python erabiltzen dute kode-adibidetan.
- Jupyter Notebookek** — esperimentazioa errazten dute, datu-zientzialari eta ML ikertzaileentzat oinarrizko tresnak bilakatu dira.
- Errendimendu-arazoak konponduta** — liburutegi gehienak C/C++ edo CUDA-n implementatuta daude azpian; Pythonek interfaze-geruza gisa funtzionatzen du, ez exekuzio-motor gisa.

BESTE LENGOAIAK — NOIZ ERABILTZEN DIREN

- **R** — estatistika-ikerketan eta bioinformatika sailetan. Akademian Pythonen alternatiba ohikoena. Tresna sendoak hipotesi-azterketa eta bistaratze estatistikorako (`ggplot2`, `tidyverse`).
- **Java** — enpresa-sistemetan, bereziki Big Data ingurunean. Apache Spark Scala/Java-n idatzita dago, Hadoop Java-n. Banku eta enpresa-aplikazio kritikoetan kontuan hartzen da.
- **JavaScript** — web-aplikazioetan AA integratzeko. **TensorFlow.js**-ekin, ereduak nabigatzailean exekutatzen dira inolako serverrik gabe. Pribatutasun-arrazoiengatik interesgarria.
- **NodeJS** — backend-zerbitzariak LLM-ekin integratzeko (Gemini API, OpenAI API). `async/await` patroia ondo egokitzen zaio LLM-deien latentzia luzeari.

Q Adibidea: *Hiru enpresa, hiru aukera - Startup baten datu-zientzia taldeak*

Bilbon → Python-en lan egiten du. Liburutegi guztiak eskura, lan-merkatuko talentua erraz. - **Banketxe handi batek Madrilen** → Java menderatzen du. ML eredu berriak Python-en idatzi eta gero Java-ra portatzen ditu sistema kritikoetan integratzeko. - **Bioteknologia-enpresa batek Donostian** → R erabiltzen du, ikertzaile gehienak unibertsitatetik datozelako. Datu-azterketa estatistikoan oso indartsuak dira.

💡 **Idea nagusia** Ez dago "lengoaia onenik" abstrakzioan. Lengoaia onena testuingurura egokitzen dena da: enpresaren ekosistema, taldearen ezagutza, proiektuaren errendimendu-eskakizuna eta hedapen-helburuak. Beti galdetu "zertarako?" lengoaia aukeratu baino lehen.

⚠ **Kontuz: "Pythonek hau guztia egin dezake" mitoa** Denbora errealeko sistemetan errendimendua kritikoa denean (1 ms-eko latentzia muga), C++ edo Rust gomendagarriagoak dira. Python ez da panazea, bere mugak ere baditu.

❑ **Hausnartzeko** Zure lehen lan-postuan, jakingo bazenu enpresak Java erabiltzen duela bere sistema kritikoetan eta Python datu-zientziarako, nola prestatuko zinateke? Biak ikaskoko zenituzke? Ala batean espezializatu?

❑ **Ariketa 1.2 · 20 min · taldeka Egoera:** 4 enpresetan praktikak egiteko aukera duzu. Bakoitzak teknologia ezberdina erabiltzen du:

1. **Glovo (Bartzelona)** — Python + JavaScript
2. **BBVA (Madril)** — Java + Python
3. **Roche (Basilea)** — R + Python
4. **Vodafone (Madril)** — Java + NodeJS

Eginkizuna: Hautatu enpresa bat eta arrazoitu zergatik. Zein lengoaiak ikasi beharko zenituzke aurretik? Zein liburutegi nagusi? Aurkeztu erantzuna ikaskideei.

1.3 Markatze-lengoaiak: XML, HTML, JSON, YAML

Markatze-lengoaiak datuak antolatzeko eta egituratzeko erabiltzen dira. Programazio-lengoaletatik desberdinak dira: ez dute logika-fluxurik exekutatzen, aldiz, datuak adierazten eta egituratzen dituzte. AArako garrantzitsuak dira: datu-trukea, konfigurazioa eta web-integrazioa markatze-lengoaiei bidez egiten dira. Atal honetan lau familia nagusiak konparatuko ditugu.

XML (EXTENSIBLE MARKUP LANGUAGE)

XML 1998an ezarri zen W3C-ren estandar gisa. Datuak hierarkikoki egituratzen ditu etiketen bidez. Industria askotan zabal erabilia da gaur egun (banku-sistemak, dokumentu-trukea, SOAP web-zerbitzuak), nahiz eta JSON-ek ordezkatu duen hein handi batean.

```
<?xml version="1.0" encoding="UTF-8"?>
<bezeroak>
  <bezeroa id="001">
    <izena>Ane Etxebarria</izena>
    <email>ane@firma.eus</email>
    <erosketak>
      <erosketa data="2024-09-15" prezioa="89.50"/>
    </erosketak>
  </bezeroa>
</bezeroak>
```

Ezaugarriak: hierarkikoa, sintaxi zorrotza, astuna parseatzeko eta luzea idazteko, baina sektore tradizionalan (bankua, gobernua) bizirik dirau.

HTML (HYPERTEXT MARKUP LANGUAGE)

HTML-k web-orrien egitura definitzen du. XMLaren ahizpa da: sintaxia antzekoa baina ez hain zorrotza (akatsen aurrean tolerantia). Web-aplikazioak eraikitzeke ezinbestekoa.

```
<!DOCTYPE html>
<html lang="eu">
<head><title>Nire AA aplikazioa</title></head>
<body>
  <h1>Galdetu Gemini-ri</h1>
  <input type="text" id="galdera">
  <button onclick="bidali()">Bidali</button>
</body>
</html>
```

JSON (JAVASCRIPT OBJECT NOTATION)

JSON XMLaren oinordeko nagusia da datu-trukerako. Arinagoa, irakurterazagoa eta API gehienetan (REST APIs) lehenetsia. Gemini API-k, OpenAI API-k eta beste edozein LLM API modernoek JSON erabiltzen dute datu-trukean.

```
{
  "bezeroa": {
    "id": "001",
    "izena": "Ane Etxebarria",
    "erosketak": [
```

```
{ "data": "2024-09-15", "prezioa": 89.50 }
}
```

YAML (YAML AIN'T MARKUP LANGUAGE)

YAML JSONaren bertsio "human-friendly" bat da. Indentazioa erabiltzen du hierarkia adierazteko (Python-en antzera), eta iruzkinak onartzen ditu. ML-konfigurazioetan, Docker Compose-en eta GitHub Actions-en oso erabilia.

```
bezeroa:
  id: "001"
  izena: Ane Etxebarria
  erosketak:
    - data: 2024-09-15
      prezioa: 89.50
```

LAU MARKATZE-LENGOAIEN KONPARAKETA

Ezaugarria	XML	HTML	JSON	YAML
Erabilera nagusia	Dokumentuak, trukea	Web-orriak	API trukea	Konfigurazioak
Sintaxiaren zorrotasuna	Oso zorrotza	Tolerantzia	Zorrotza	Indentazio-mendekoa
Irakurteraztasuna	Ertaina	Ertaina	Ona	Bikaina
Iruzkinak	Bai	Bai	Ez	Bai
Pisua	Astuna	Ertaina	Arina	Oso arina
Maiztasuna AAn	Gutxi	Ertaina (web)	Handia	Handia

Marko-lengoaiei konparaketa

XML	HTML	JSON	YAML
<code><tag>balioa</tag></code>	<code><div>edukia</div></code>	<code>{"giltza": "balioa"}</code>	<code>giltza: balioa</code>
- Hierarkikoa - Sintaxi zorrotza - Tradiziozkoa	- Web-orriak - Tolerantzia - Bisuala	- APIak - Arina - IAren estandarra	- Konfigurazioak - Irakurterraza - Iruzkina

Bakoitzak bere lekua dauka

3. Irudia: XML, HTML, JSON eta YAML formatuen konparaketa-koadroa, kasu-erabileraren arabera.

Q Adibidea: Gemini API-ari deia Gemini-ri galdera bat egitean, API-ari JSON bidaltzen diozu eta JSON jasotzen duzu erantzun gisa. Beraz, JSON ezagutzea ezinbestekoa da edozein LLM-aplikaziorekin lan egiteko. Adibidez: `json {"model": "gemini-2.5-flash", "contents": "Kaixo!"}`

! Ideia nagusia Lau markatze-lengoaia osagarriak dira, ez aurkari. Egoera bakoitzeko aukerarik egokienak lirateke:

konfigurazioak → YAML,

datuak → JSON,

web orriak → HTML,

dokumentu zaharrak → XML.

□ **Ariketa 1.3 · 15 min · banaka Egoera:** Gemini API-tik 5 bezeroren erosketareregistroa jaso duzu JSON formatuan. **Eginkizuna:** Bihurtu erregistro hori YAML formatura paperean (5 minutu maximo). Gero, idatzi galdera bat erantzuteko: "Zer egingo zenuke datuak XML formatuan jaso bazenitu? Zergatik JSON-ek dauka abantaila?"

Demagun JSON hau:

```
{
  "bezeroak": [
    {"id": "001", "izena": "Ane", "erosketa": 89.50},
    {"id": "002", "izena": "Mikel", "erosketa": 23.10},
    {"id": "003", "izena": "Leire", "erosketa": 145.00},
    {"id": "004", "izena": "Jon", "erosketa": 12.75},
    {"id": "005", "izena": "Maite", "erosketa": 67.90}
  ]
}
```

2. Ingurune Profesionala

Atal honetan benetako lan-fluxua osatzen duen kodea idazteko ingurune profesionala landuko da: editorea (Antigravity), ingurune birtualak (proiektuen isolamendua), pakete-kudeaketa (`pip` eta `PyPI`) eta dependentzien dokumentazioa (`requirements.txt`). Lau elementu hauek elkarrekin lan egiten dute, eta beharrezkoak dira edozein lankidetzat proiektu (kolaboratibo) eraikitzeko.

2.1 Antigravity (eta VS Code-ren oinordetza)

Edozein garatzailek ordu asko ematen ditu egunero kode-editorean. Tresna hori ondo ezagutzeak eta egokitzeak alde handia egiten du produktibitatean eta akatsen prebentzioan. Modulu honetan **Google Antigravity** erabiliko dugu lehenetsi gisa, bi arrazoiengatik: ikastetxeak Google lizentzia duela, eta AA agentearen aroari egokituta dagoela jaiotzazko diseinuz.

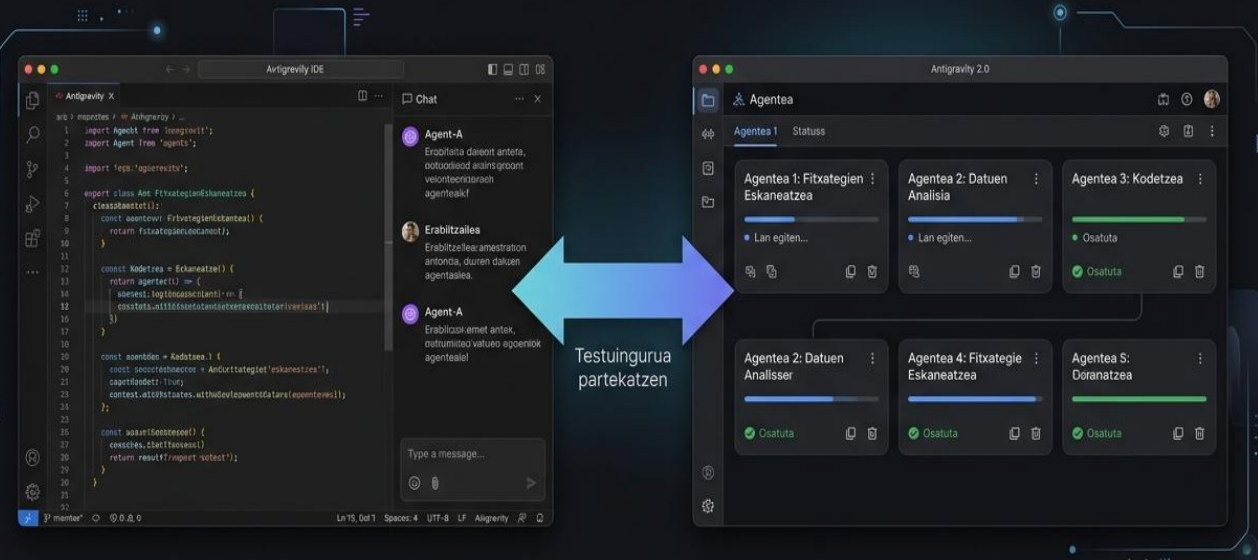
ZER DA ANTIGRAVITY

Antigravity Google-k 2025eko azaroan kaleratu zuen garapen-plataforma da, Gemini 3 ereduarekin batera. VS Code-ren kode-oinarria hartu eta sakonki birmoldatu zuen Google-k, AA-agenteen koordinazioan zentratuta. Editorearen funtsezko portaera (fitxategi-esploratzailea, terminala, debugger-a, Git integrazioa) VS Code-ren parekoa da. 2026ko maiatzean, Google I/O-n, Antigravity 2.0 kaleratu zen: IDE-a agente-sistema bikoitz batean bilakatu zen, non IDE-a eta mahaigaineko aplikazio independente bat elkarrekin funtzionatzen duten.

Aginte Bikoitzeko Modua (Dual-Agent System):

- **Antigravity IDE** → kode-editorea agente-alboko barra batekin. Garatzaileak kode-laguntzailearekiko ohiko elkarrizketa mantentzen du, inline komandoak erabiltzen ditu (`Ctrl+I`), eta kode-proposamenak onartzen ditu (`Tab`). VS Code-ren fork bat da, baina Open VSX Registry erabiltzen du Microsoft Marketplace-aren ordez.
- **Antigravity 2.0** (Desktop App) → mahaigaineko Electron aplikazio independentea, agente-orkestratzaile gisa funtzionatzen duena. Hainbat azpi-agente paraleloan kudeatu, zeregin programatuak (cron) konfiguratu, eta proiektu desberdinak gainbegiratu ditzakezu. IDE-tik kanpo exekutatzen da, baina proiektuaren testuingurua partekatzen du.

Aginte Bikoitzeko Modua



Antigravity IDE — kode-editorea
agente-alboko barrarekin

Antigravity 2.0 —
mahaigaineko orkestratzailea

4. Irudia: Antigravity-ren Aginte Bikoitzeko Modua: Antigravity IDE (ezkerrean, kode-editorea agente-alboko barra batekin) eta Antigravity 2.0 Desktop App (eskuinean, mahaigaineko orkestratzaile independentea, azpi-agentek paraleloan kudeatzen).

Onartutako ereduak:

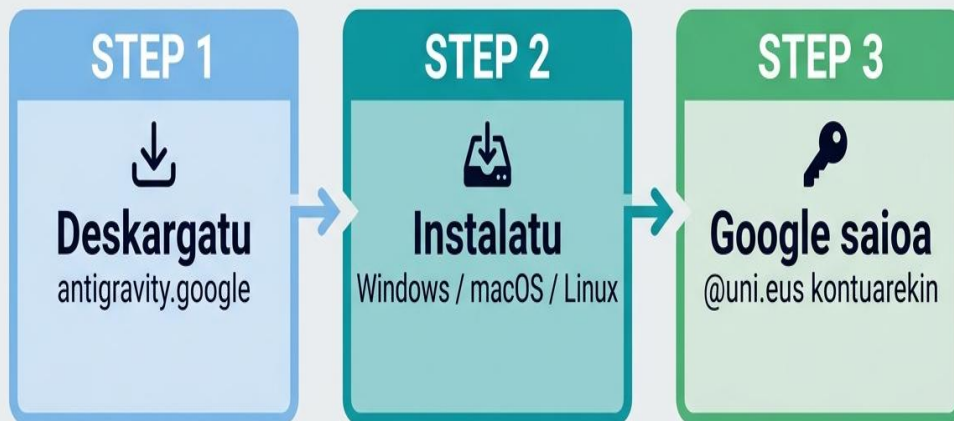
Eredua	Hornitzailea	Erabilera tipikoa
Gemini 3.5 Pro	Google (berriena)	Hausnarketa aurreratua, agenteen orkestrazioa
Gemini 3.1 Pro	Google	Oraindik erabilgarria, hausnarketa konplexua
Gemini 3 Flash	Google	Azkarragoa eta merkeagoa, eguneroko lana
Claude Sonnet 4.6 / Opus 4.6	Anthropic	Alternatiba, kode-azterketa sakona
GPT-OSS-120B	OpenAI	Kode irekiko bertsioa, esperimentuetarako

INSTALAZIOA

[Antigravity](#) webgune ofizialetik deskarga daiteke Windows, macOS eta Linux sistemetarako. Instalazioaren ondoren, Google kontuarekin saioa hasi behar da (ikastetxeko @uni.eus kontuak baliodunak dira).

Alternatiba gisa, VS Code ere erabil daiteke — funtsezko ezaugarriak partekatzen dituzte. Hala ere, bi desberdintasun garrantzitsu daude: (1) Antigravity-k Open VSX Registry erabiltzen du luzapenatarako, eta (2) Pylance ezin da Antigravity-n erabili (BasedPyRight erabiltzen da ordez). [VS Code](#)

Antigravity instalatzea: hiru urrats



Doan, ikastetxearen lizentziarekin

5. Irudia: Antigravity instalatzeko hiru urratsak: deskargatu, instalatu, Google kontuarekin saioa hasi.

LUZAPEN GARRANTZITSUENAK

Antigravity (eta VS Code) instalatu berri daudenean ez dute Python "hasieratik" ezagutzen. Luzapenek bihurtzen dute editorea adimendun:

- **Python (Microsoft)** — IntelliSense, linting, debugging eta testing.
- **BasedPyRight** — mota-egiaztapena (type checking) eta osatze aurreratua. Pyright-en fork irekia da. Antigravity-n erabiltzen da Pylance-ren ordez, azken honen lizentzia Microsoft-en

produktuetara mugatuta dagoelako (ezin da VS Code-ren fork batean erabili). VS Code-n, berriz, Pylance zuzenean erabiltzen da lehenetsi gisa.

- **Black Formatter** — kode-formateatzaile automatikoa (PEP 8).
- **GitLens** — Git historia editorean bertan.
- **Jupyter** — Jupyter notebookak editorearen barruan.
- **autoDocstring** — funtzioen dokumentazio-txantiloak automatikoki.
- **Flake8** — funtzioen dokumentazio-txantiloak automatikoki.

ANTIGRAVITY EKOSISTEMA: LAU GAINAZAL

Antigravity 2.0-tik aurrera, plataforma lau gainazaletan banatzen da, bakoitza erabilera-kasu desberdinetarako:

- Antigravity IDE → kode-editorea, VS Code-ren fork-a. Kodea zuzenean editatu, inline komandoak erabili (Ctrl+I), eta agente-alboko barran elkarrizketa mantendu.

📱 Antigravity 2.0 (Desktop App) → mahaigaineko Electron aplikazioa. Agente-orkestratzailea: azpi-agenteak kudeatu, zeregin programatuak (cron) konfiguratu, proiektuak gainbegiratu.

💻 Antigravity CLI (agy) → terminaleko interfazea. Zerbitzari buruguneetan (headless), SSH bidez, eta automatizazio azkarrean erabiltzeko.

- Antigravity SDK → Python liburutegia. Agente pertsonalizatuak programatzeko, tresna berriak gehitzeko, eta lankidetzaz-fluxuak eraikitzeko.

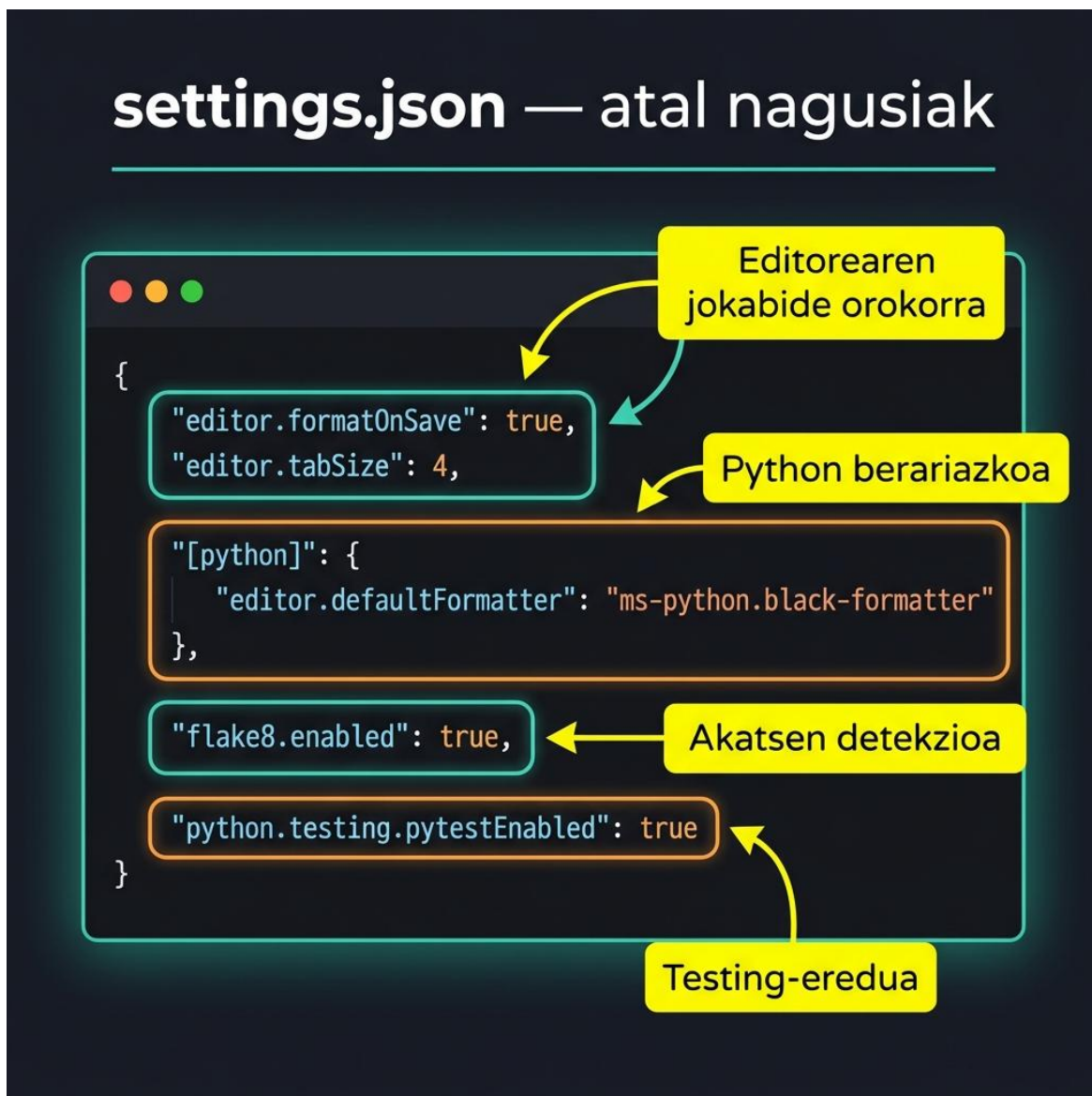
KONFIGURAZIO GOMENDATUA

`settings.json` fitxategian gordetzen da konfigurazioa. Honako hau ondo dabil proiektu gehienetan:

```
{
  "editor.formatOnSave": true,
  "editor.tabSize": 4,
  "editor.insertSpaces": true,
  "editor.rulers": [
    88
  ],
  "files.trimTrailingWhitespace": true,
  "files.insertFinalNewline": true,
  "[python]": {
    "editor.defaultFormatter": "ms-python.black-formatter",
    "editor.formatOnSave": true
  },
  "flake8.enabled": true,
  "python.testing.pytestEnabled": true,
```

```
"editor.minimap.enabled": false,
"workbench.colorTheme": "Default Dark Modern",
"workbench.auxiliaryActivityBar.location": "default",
"python.languageServer": "default"
}
```

△ **Oharra:** "default" balioak BasedPyRight aktibatzen du Antigravity-n, baina Pylance aktibatzen du VS Code-n. Hau lizentzia-desberdintasunagatik da.



6. Irudia: Antigravity-ren settings.json fitxategiaren atal nagusiak anotatuta — editor-konfigurazioa, Python berariazkoa, linting eta testing.

LASTERBIDE GARRANTZITSUENAK

Lasterbidea	Funtzioa
Ctrl + P	Fitxategiak bilatu
Ctrl + Shift + P	Komandoen paleta (<code>settings.json</code> ere bertan bilatu)
Ctrl + `	Terminala ireki/itxi
F5	Debuggerra hasi
F9	Breakpoint jarri/kendu
Ctrl + D	Hitz bera bilatu eta hautatu
Alt + ↑/↓	Lerroa gora/behera mugitu
Ctrl + /	Lerroa komentatu
Ctrl + Shift + K	Lerroa ezabatu
Ctrl + I	Agente bati galdera bat egin (Antigravity)
Ctrl + Shift + M	Antigravity 2.0 Mahaigaineko Aplikazioa / Orkestratzailea ireki (Aginte Bikoitzeko Modua)

LINTING ETA FORMATEATZE TRESNAK

Kalitate-tresnak bi multzo nagusitan banatzen dira: **linter-ak** (akatsak detektatu kodea exekutatu aurretik) eta **formateatzaileak** (kodearen itxura estandarizatu). Praktika profesionalean biak konfiguratzen dira fitxategia gordetzean automatikoki exekuta daitezen.

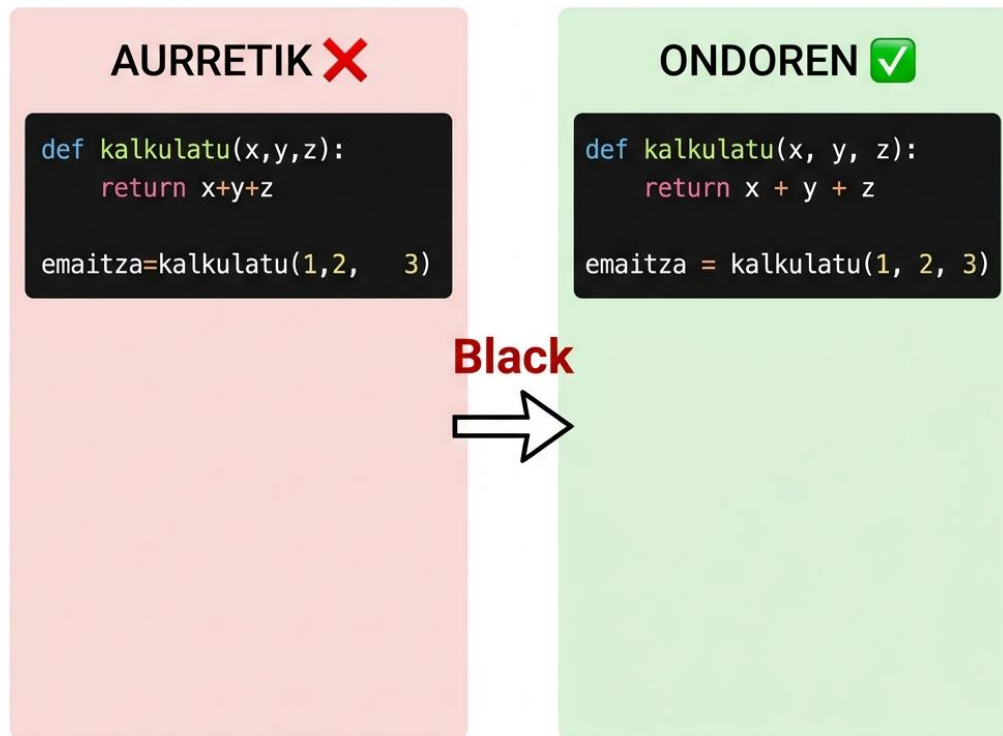
```
# AURRETIK (formateatu gabe)
def kalkulatu(x,y,z):
    return x+y+z

emaitza=kalkulatu(1,2, 3)

# ONDOREN (Black-en ondoren)
def kalkulatu(x, y, z):
    return x + y + z

emaitza = kalkulatu(1, 2, 3)
```

Black formateatzailearen lana



PEP 8 estandar automatikoki aplikatua

7. Irudia: Black formateatzailearen lana — kode itsusi bat (ezkerrean) eta automatikoki formateatutakoa (eskuinean), aldaketak nabarmenduta.

VS CODE-REN OINORDETZA

Lan-merkatuan, **VS Code** da oraindik gehien erabiltzen den editorea. Modulu honetan Antigravity-rekin ikasten duzun guztiak ere VS Code-an doi-doi funtzionatuko du:

Atala	Antigravity	VS Code
Luzapenak	Open VSX Registry (nagusia)	Microsoft Marketplace (bera)
Konfigurazioa	settings.json	settings.json (formatu bera)
Lasterbideak	Goian erakutsiak	Berdinak
Git integrazioa	Natiboa	Natiboa (bera)
AA-laguntza	Natiboa (Aginte Bikoitzeko Modua: IDEa + Desktop App orkestratzailea)	Luzapenen bidez (GitHub Copilot, Gemini Code Assist)
Pylance	Ez (lizentzia mugatua)	Bai (lehenetsia)

Beraz, edozein enpresan VS Code aurkitzen baduzu, segidan funtzionatzen duzu — ezagutza guztiak transferitu egiten dira.

Q Adibidea: Aginte Bikoitzeko Modua (Dual-Control Mode) praktikan Demagun proiektu batean ari zarela: web aplikazio txiki bat eraikitzen FastAPI-rekin. Aginte Bikoitzeko Moduan, kodea IDEan idazten duzun bitartean, Antigraity 2.0 Mahaigaineko Aplikaziotik hiru agente (Subagents) orkestratzen dituzu aldi berean: bati esaten diozu testak idatzi ditzala, beste bati dokumentazioa eguneratzeko, eta hirugarrenari errore-kudeaketa hobetzeko. Hirurek paraleloan lan egiten dute atzeko zereginetan. 10 minutu igarotzen dira eta hirurak bukatuta daude. IDEan bertan berrikusi visual diff-ekin, onartu edo aldatu. Aurretik lan-bolumen horretarako goiz osoa beharko zenuke.

💡 Ideia nagusia Antigraity ez da soilik editore bat: garapen-fluxuaren orkestrazio-plataforma bat da. AA agenteak ez ditu **ordezkatz**en garatzaileak — bere produktibitatea handitzen du atazak banatzen eta paraleloan egiten utziz. "Human in the loop": gizakiak beti hartzen du azken erabakia.

⚠ Kontuz: Kode automatikoki sortua ez da itsuki onartu behar! Agenteak kode sintaktikoki zuzena idaztzi arren ez dauka **proiektuaren testuingurua** osoa.

Beti berrikusi:

- Aldagai-izenak: bat datoz proiektuaren konbentzioarekin?
- Liburutegiak: hauek dira proiektuan baimendutakoak?
- Logika: benetan egiten du eskatutakoa, edo aproposa "iruditzen zaio" agenteari?
- Segurtasuna: kontuz hardcoded API-gakoekin edo SQL injection-ekin.

☐ **Hausnartzeko** Eragin buruari: imajinatu programatzaile bat 2020an (agenterik gabe) eta 2026an (agenteekin). Zenbat denbora ematen du erronkak ulertzeko, kodea idazteko eta probatzeko? Zer aldatzen da bere lanean? Zer ez da aldatzen?

☐ **Ariketa 2.1 • 30 min • banaka Egoera:** Antigraity ez duzu sekula erabili. Praktikaldiko tutoreak lehen erronka ezarri dizu: instalatzea eta lehen agentea probatzea!

Eginkizunak:

1. Antigraity instalatu eta Google kontuarekin saioa hasi.
2. Python (Microsoft) luzapena instalatu.
3. `settings.json` editatu goian agertzen den konfigurazioarekin.
4. Karpeta berri batean Python fitxategi sinple bat sortu (`kaixo_mundua.py`).
5. Antigraity IDE-an, agente-alboko barra ireki (Ctrl+I) eta eskatu funtzioa sortu dezala 3 parametroekin (izena, adina, herria) eta agur pertsonalizatua itzul dezala.
6. Antigraity 2.0 (Desktop App) ireki, proiektu bera kargatu, eta elkarrizketa berri batean eskatu testak sortzeko funtzio horretarako (`test_agur.py`). Bi agenteak (IDE-koa eta Desktop App-ekoa) paraleloan lan egiten ikusi.

☑ **Entrega:** bi fitxategiak (agur eta testak), 100 lerro baino gutxiagorekin. Aginte Bikoitzeko Moduaren screenshot-ak: IDE-ko agente-alboko barra eta Antigravity 2.0-ko zeregin-panela erakutsiz.

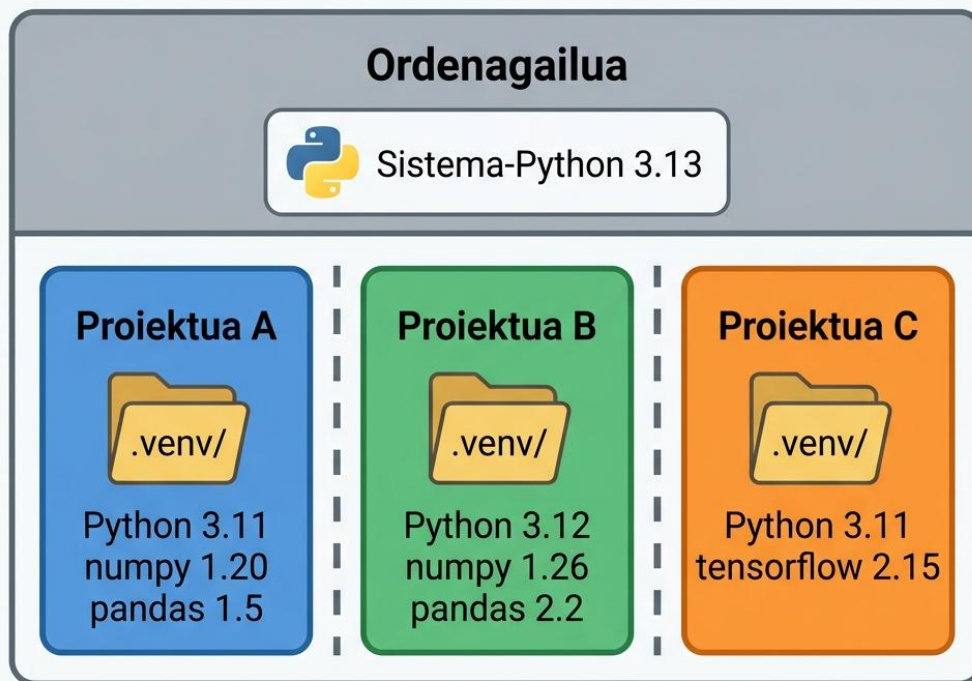
INGURUNE BIRTUALEEN EZARPENA: ZERGATIK DIRA BEHARREZKOAK

Python proiektuak ez dira modu isolatuan existitzen: bakoitzak hainbat kanpoko liburutegi erabiltzen ditu (NumPy, Pandas, scikit-learn), eta liburutegi horiek etengabe aldatzen dira bertsio berriekin. Hori da **dependentzia-infernuaren** (dependency hell) erroa: proiektu desberdinek liburutegi beraren bertsio desberdinak behar dituzte, eta sistema bakarrean ezin dira aldi berean instalatu.

Proiektua A: numpy==1.20 behar du
Proiektua B: numpy==1.26 behar du

Ingurune birtualek proiektu bakoitzari bere Python eta paketeak isolatzeko aukera ematen diote. Funtsean karpeta bat da, bere Python interprete propioarekin eta `site-packages` propioarekin.

Ingurune birtualak: proiektuen isolamendua



Bakoitzak bere 'Python txikia' du, sistema globala ukitu gabe

8. Irudia: Ingurune birtualen isolamendua bisualki — proiektu bakoitzak bere "Python txiki" propioa du, sistema globala ukitu gabe.

VENV (PYTHON ESTANDARRA)

Natiboki dator Pythonekin batera eta ez dago instalatu beharrik.

```
# 1. Karpeta ingurune birtuala sortu (karpeta sortuko du)
python -m venv .venv

# 2. Aktibatu
# Windows (PowerShell)
(Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass)
.venv\Scripts\Activate.ps1
# Linux / macOS
source .venv/bin/activate

# 3. Aktibatuta dagoela egiaztatu — terminal-ean (.venv) agertuko da berdez:
# (.venv) C:\proiektua>

# 4. Paketeak instalatu (ingurune birtualean bakarrik)
pip install numpy pandas

# 5. Desaktibatu
deactivate
```

CONDA (ALTERNATIBA)

conda Anaconda/Miniconda pakete-kudeatzaileak eskaintzen duen sistema zabalagoa da: Python paketeak ez ezik, beste lengoaietako liburutegiak (C, C++, R) ere kudeatu ditzake. Data Science eta Machine Learning proiektuetan bereziki erabilgarria da.

[Miniconda](#) bertsio murriztuarekin nahikoa da Pythonen bertsio desberdinak instalatzeko eta beharrezkoak diren paketeak, eskuragarri haien webgune nagusitik.

```
# Ingurune berria sortu Python bertsio zehatzarekin
conda create -n proiektua python=3.11

# Aktibatu
conda activate proiektua

# Pakete bat instalatu
conda install numpy
```

VENV VS CONDA

Ezaugarria	venv	conda
Instalazioa	Python-ekin dator	Bereiztuta instalatu behar
Python bertsioa aukeratu	✗ Ez	✓ Bai
Python ez diren liburutegiak	✗ Ez	✓ Bai (CUDA, MKL...)
DS/ML proiektuak	☐ Nahikoa	✓ Gomendatua
Web proiektu hutsak	✓ Gomendatua	☐ Gehiegizkoa

Q Adibidea: bertsio-gatazka tipikoa Bi proiektutan zaude lanean: A-k TensorFlow 2.10 behar du; B-k 2.15. Ingurune birtualik gabe, A instalatzen duzunean B ez da exekutatzen; B instalatzen duzunean A ez da exekutatzen. Ingurune birtualekin: bi karpeta ezberdin, bi Python ezberdin, bi TensorFlow bertsio aldi berean instalatuta. Bi proiektuek funtzionatzen dute beren ingurunean.

! Ideia nagusia Ingurune birtualak proiektu bakoitzaren "Python txiki" propioak dira. Inoiz ez instalatu packageak sistema-mailan (`sudo pip install` debekatuta dago praktika onetan); beti aktibatu ingurune birtuala lehenik.

⚠ Kontuz: .venv ez igo Git-era .venv karpeta .gitignore-an sartu behar da inoiz ez igotzeko Git biltegiara. Karpeta hori astuna da (askotan GB-ak) eta proiektuen arteko erreproduzigarritasuna `requirements.txt`-rekin lortzen da, ez ingurune birtuala kopiatuz.

□ **Ariketa 2.2 · 15 min · banaka**

Eginkizuna:

1. Antigravity-n karpeta berri bat sortu (`numpy-proba/`).
2. Terminalean, `venv` ingurune birtuala sortu eta aktibatu.
3. `numpy` instalatu (`pip install numpy`).
4. Python script bat sortu, `import numpy as np` egiten duena eta array bat inprimatzen duena. Exekutatu.
5. Desaktibatu ingurunea (`deactivate`). Berriz exekutatu scripta: errorea jasoko duzu? Zergatik?

2.3 pip eta Pakete Kudeaketa

Python-en indar handi bat bere ekosistema da: milaka liburutegi irekiak, edozein arazori soluzioa eskaintzen diotenak. Liburutegi horiek **PyPI** (Python Package Index) izeneko biltegi zentrolean argitaratzen dira. `pip` Python-en package-kudeatzaile ofiziala da.

```
# Pakete bat instalatu
pip install numpy

# Bertsio zehatz bat instalatu
pip install numpy==1.26.4

# Bertsio-tarte bat instalatu
pip install "numpy>=1.24,<2.0"

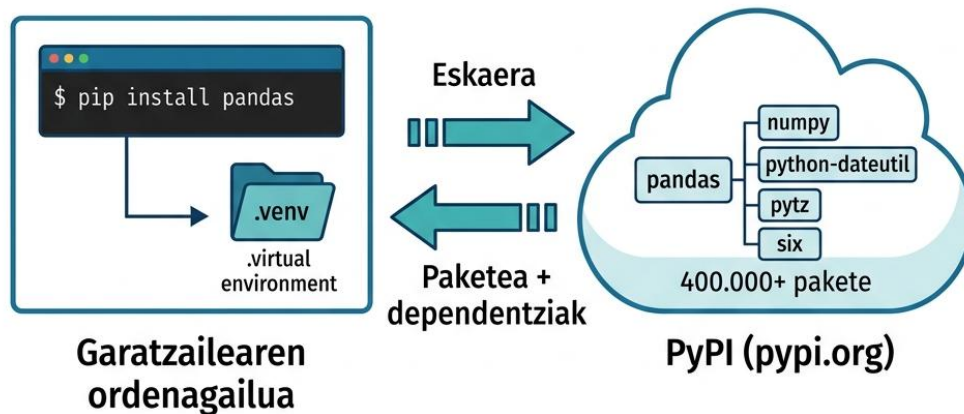
# Pakete bat eguneratu
pip install --upgrade numpy

# Pakete bat desinstalatu
pip uninstall numpy
```

```
# Instalatutako paketeak ikusi
pip list

# Pakete baten informazioa ikusi
pip show numpy
```

pip install – PyPI-tik garatzailearen ingurunera



Dependentzia-ebazpena automatikoki kateatua

9. Irudia: `pip install` komandoaren fluxua — garatzailearen ordenagailutik PyPI biltegira eta itzulera dependentziekin.

Pakete bat instalatzen denean, `pip`-ek paketearen metadatuak irakurri eta bere **dependentziak** ere automatikoki deskargatzen ditu. Adibidez, `pip install pandas` exekutatzean, `pandas`-ez gain `numpy`, `python-dateutil`, `pytz` eta beste hainbat pakete instalatzen dira modu errekursiboan.

Q Adibidea: `pip install pandas` Komando bakar bat — bost edo sei paketeko deskarga kateatua. PyPI biltegitik metadatuak hartzen ditu `pip`-ek, dependentzia-grafoa eraikitzen du, eta dena instalatzen du ordena egokian. Hiruzpalau segundo gehienetan. Tradiziozko software-instalazioarekin alderatuta (CD-ak, asistenteak, errore-mezuak) hau aurrerapen erraldoia da.

! **Idea nagusia** *pip* ez da soilik instalazio-tresna: *dependentzia-ebazpenaren eta -kudeaketaren motor osoa da. Hori da Python-en ekosistemaren indarra eta arrisku nagusietakoa aldi berean: dependentzia infinituen kateak.*

□ **Ariketa 2.3 · 10 min · banaka Eginkizuna:** Ingurune birtual berri batean instalatu honako paketeak banan-banan, eta *pip list* exekutatu instalazio bakoitzaren ondoren: *requests, pandas, scikit-learn*. Zenbat pakete dauzkazu ingurunean bakoitzaren ondoren? Zergatik gehitzen dira hainbeste?

2.4 requirements.txt

Software-ingeniaritzaren kontzeptu funtsezko bat **erreproduzigarritasuna** da: gaur ordenagailu batean funtzionatzen duen kodeak biharko ordenagailu batean ere funtzionatu behar du. *requirements.txt* proiektuaren dependentzia zehatzen "errezeta-fitxa" da.

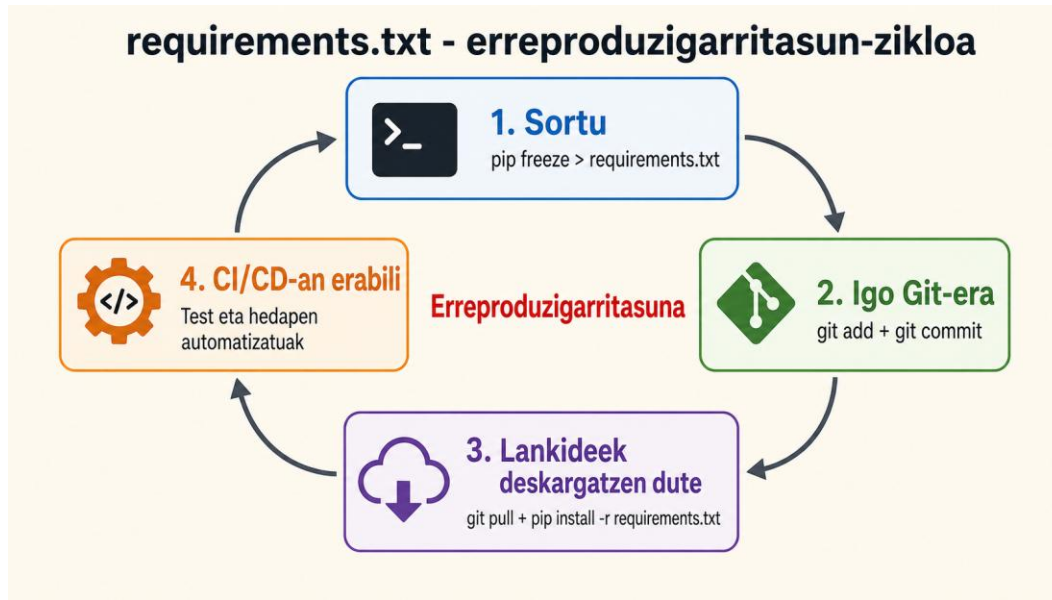
```
# Ingurune birtualean instalatutako guztia esportatu
pip freeze > requirements.txt
```

Honek hau bezalakoa sortzen du:

```
certifi==2024.2.2
numpy==1.26.4
pandas==2.2.1
python-dateutil==2.9.0
pytz==2024.1
requests==2.31.0
scikit-learn==1.4.0
```

Beste garatzaile batek edo zerbitzari batek proiektua martxan jartzeko:

```
# Ingurune berri batean dena instalatu
pip install -r requirements.txt
```



10. Irudia: requirements.txt fitxategiaren bizi-zikloa — sortu (`pip freeze`), Git-era igo, lankideak instalatu (`pip install -r`), CI/CD pipelinetan erabili.

PRAKTIKA ONAK

1. Bertsio finkoak (==) vs malguak (>=)

Produkzioan beti finkatu bertsioak (`numpy==1.26.4`); garapenean malgutasun pixka bat onargarria da (`numpy>=1.24`).

2. requirements fitxategi anitz

```

requirements/
├── base.txt      # Oinarrizko dependentziak
├── dev.txt       # Garapen tresnak (pytest, black)
└── prod.txt      # Produktzio-bakarrak (gunicorn)
    
```

Q Adibidea: Maria deialdiei aurre egiten Mariak script bat utzi du baina ez du `requirements.txt`-rik. Zuk Python eta `pandas` instalatu dituzu. Scriptak ez du funtzionatzen: "ModuleNotFoundError: No module named 'openpyxl'". `pip install openpyxl` egiten duzu. Aurrera. "ModuleNotFoundError: No module named 'requests'". Paketez pakete, 15 minutu pasatzen dira. Mariak `pip freeze > requirements.txt` eginda zukeen 30 segundotan eta zuk `pip install -r requirements.txt` egingo zenukeen 1 minutuan. **Erreproduzigarritasuna ez da luxua, oinarrizko ohitura da.**

💡 Ideia nagusia `requirements.txt` Git biltegira igo behar da, **ingurune birtuala** ez. Lankideek beren ingurunea sortuko dute zure `requirements.txt`-tik, eta ekosistema identikoa edukiko dute.

⚠ Kontuz: bertsio gabe arazoak `requirements.txt`-ean bertsiorik gabe (`numpy` soilik, ez `numpy==1.26.4`), urtebete geroago instalatzean Python-eko liburutegiek bertsio berriak

izan ditzakete eta API-aldaketek hauts dezakete zure kodea. Beti finkatu bertsioak produkzioan.

□ Ariketa 2.4 • 20 min • taldeka

Egoera: Taldekide batek hau bidali dizu: bere proiektuko `main.py` (Pandas erabiliz CSV bat irakurri eta laburpena ematen duena) eta `requirements.txt` (5 lerro).

Eginkizuna:

1. Ingurune birtual berria sortu, `requirements.txt`-tik instalatu.
2. Scripta exekutatu CSV-arekin.
3. Aldatu scripta (gehitu zutabe berri bat).
4. Eguneratu `requirements.txt`-a pakete berri batekin (matplotlib).
5. Bidali atzera taldekideari aldatutako bertsioa, eta egiaztatu berak ere funtzionatzen duela.

2.5 Agenteak eta AArekiko praktika onak

Antigravity instalatuta daukazu (§2.1), ingurune birtuala konfiguratuta, paketeak ondo. Baina **AA agente bat ondo erabiltzen ez badakizu**, tresnak ez du balio handirik: prompt lausoak idatziz erantzun lausoak jasoko dituzu, agentearen iruzkin oker bat onartzeagatik ordu osoak gal ditzakezu, eta urtea aurrera doala "vibe coding" ohitura txarrak sortuko dituzu nondik datozen jakin gabe.

Atal honetan AA agentearekin lan egiteko **sei ohitura nagusiak** ikasiko dituzu. Hemen sortutako ohiturek modulu osoaren gainerako gaietan lagunduko dizute — datu-zientzia (2. gaia), AA programazioa (3. gaia), konbergentzia teknologikoa (4. gaia) eta agente korporatiboak (5. gaia) —. Agentearen erabiltzaile on bihurtu, ez pasibo.

□ **Zergatik gaur eta ez azken gaian** Tentazioa egon daiteke "agenteen teoria" 5. gaira atzeratzea — han ikusiko baitira agente korporatiboak sakon. Baina ordurako 180 ordu pasatuko dituzu agentea **gaizki erabiltzen**, eta ohitura okerrak finkatuko zaizkizu. Hobe da gaur, lehen astetik, oinarri onak ezartzea: gerora aldatzea askoz zailagoa baita.

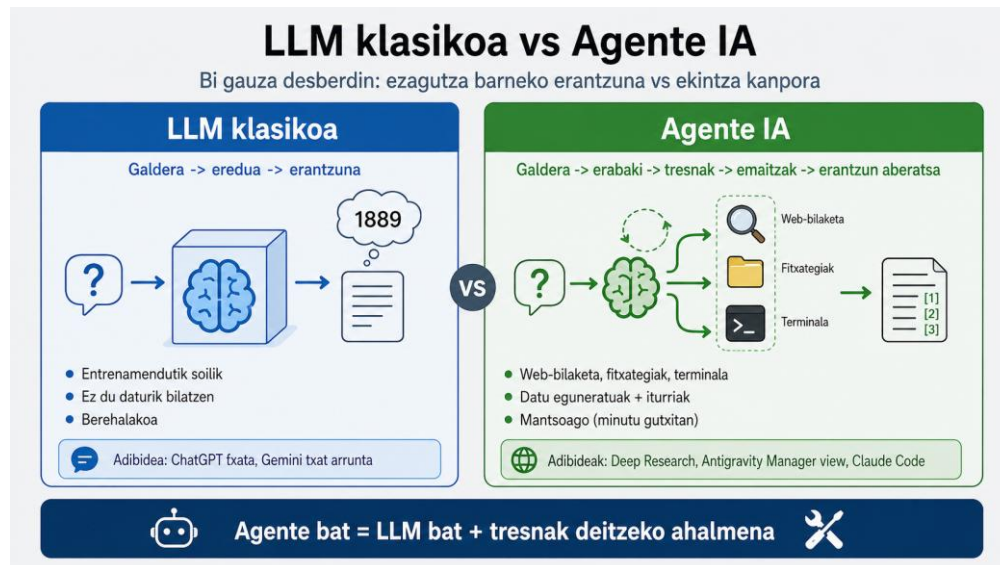
ZER DA AA AGENTE BAT? — LLM KLASIKOA VS AGENTEA

Aurrera jarraitu baino lehen, kontzeptua argitu. Askok nahasten dituzten bi gauza dira **LLM klasiko bat** eta **AA agente bat**, eta diferentzia ulertzea funtsezkoa da hurrengo guztia ulertzeko.

LLM klasiko bat (Gemini, GPT, Claude *txatera dei egiten dudanean bakarrik*): galdera bat egiten diozu, eta entrenamenduan ikasitakoa erabiliz erantzuten du **bertan eta orduan**. Ez du daturik bilatzen, ez du fitxategirik irekitzen, ez du komandorik exekutatzen. Sarrera → eredu → irteera. Kitto.

AA agente bat: galdera bat egiten diozu, eta ereduak **erabakitzen du zer egin behar duen**: agian web-bilaketa egin, agian fitxategi bat ireki, agian komando bat exekutatu, agian beste agente bati galdetu. Ekintza horiek egiten ditu (askotan zuk ikusi gabe), emaitzak prozesatzen ditu, eta gero

erantzun argitua ematen dizu. Sarrera → eredia erabakitzen du → tresnak deitzen ditu → emaitzak biltzen ditu → irteera **askoz aberatsagoa eta eguneratuagoa**.



34. Irudia: Ezkerrean LLM klasikoa zerbitzatzen erantzun bat entrenamendutik bertatik; eskuinean agente bat web bilaketa egiten, fitxategiak irakurtzen eta gero erantzuten.

□ **Adibide intuitiboa: Deep Research** Ez bazaude ziur "agente bat" erabili duzun, segur aski **Deep Research** delakoa erabili duzu: Gemini-n, ChatGPT-n edo Claude-n agertu den ezaugarri bat. Hau gertatzen da bere barruan: - **LLM hutsa**: "Noiz eraiki zuten Eiffel dorrea?" → "1889an" (entrenamendutik). - **Agentea (Deep Research)**: "Konparatu Euskadiko 5 banku nagusien hipoteka-tasak 2026ko ekainean" → modeloak ez daki erantzun zuzena (ez du ezagutza eguneratua eta enpresa-mailako konparaketak ez dakizki buruz). Beraz: **20-30 web-bilaketa egiten ditu, web orriak irakurtzen ditu, taulak ateratzen ditu, sintetizatzen du, eta dokumentu landu bat ematen dizu iturri-erreferentziekin**. Hori dena minutu gutxitan. "Deep Research" hori agente bat da, soilik web-bilaketara mugatua. Antigraity-ren agentea zabalagoa da: ez bakarrik webean bilatzeko gai da, baita zure fitxategiak irakurtzeko, kodea idazteko, terminala erabiltzeko ere.

Konparaketa praktikoa:

Ezaugarria	LLM klasikoa	AA Agentea
Erantzuna nondik	Entrenamenduko ezagutzatik	Entrenamendua + denbora errealeko ekintzak
Datu eguneratuak	Ez (geldituta data jakin batean)	Bai (bilaketa, API-deiak)
Fitxategiak / sistema	Inolaz ez	Bai (irakurri, idatzi, exekutatu)
Erantzun-denbora	Berehala (segundoak)	Mantsoago (segundo gutxitik minutuetara)
Erabilera tipikoa	Galdera-erantzun azkarra	Lan konplexua zatika
Adibide pragmatikoa	ChatGPT txat arrunta, Gemini txat klasikoa	Deep Research, Antigravity Manager view, Claude Code
Arriskua	Mugatua (datuak ez dira ateratzen)	Handiagoa (sisteman eragina dauka)

💡 **Idea nagusia** "Agentea" ez da tresna bat, gaitasun bat baizik: **LLM bat + tresnak deitzeko ahalmena**. Antigravity = LLM + tresnak (fitxategiak, terminala, web). Claude Code = LLM + tresnak (shell, fitxategiak). Deep Research = LLM + tresna bakar bat (web-bilaketa). Hortaz hurrengo ohitura guztiak (kontestua, memoria, skills, egiaztapena) **edozein tresna agentikorekin** balio dute, web-bilaketa hutsetik aitzina.

❑ **Hausnartzeko** Inoiz erabili duzu Deep Research Gemini-n edo ChatGPT-n? Konturatu zinen "agente" bat erabiltzen ari zinela? Zergatik da garrantzitsua bi mota hauek bereiztea? Eta zer ardua duzu erantzukizun-mailan: LLM klasiko batek esaten dizuna zure erantzukizunpean baduzu, agente batek **zure sistemarekin egiten duenarekiko erantzukizuna areagotzen al da?**

NOLA ENTRENATZEN DIRA LLM-AK — ETA ZERGATIK FUNTZIONATZEN DUTEN AGENTEAK

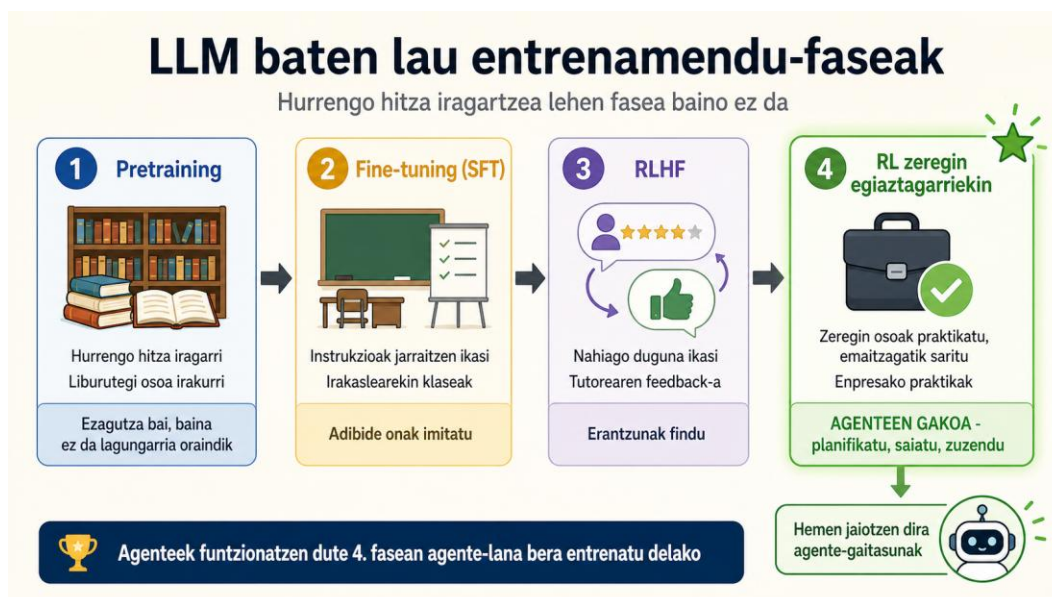
Mito zabaldu bat dago: "LLM-ak hurrengo hitza iragartzeko bakarrik entrenatu dira". Hori **lehen fasea baino ez da**. Egungo modeloei lau fase nagusi igarotzen dituzte, eta agenteen gaitasun harrigarriak —pausoz pausoko planak egitea, erroreetatik errekuaratzea, tresnak noiz erabili erabakitzea— batez ere **azken faseei** esker datoz. Eskola-ibilbide baten analogiarekin uler daiteke:

1. fasea — Pretraining (hurrengo hitza iragarri). Modeloak testu-kopuru erraldoiak irakurtzen ditu (liburuak, weba, kodea) eta gauza bakarra ikasten du: testu bat emanda, hurrengo hitza iragartzea. *Analogia: liburutegi osoa irakurri duen ikaslea — hizkuntza eta munduari buruzko ezagutza izugarria du, baina oraindik ez daki elkarrizketa bat eramaten, ezta agindu bat betetzen ere*. Fase honen ondoren modeloak testua osatzen daki, baina ez da lagungarria: galdera bati beste galdera batekin erantzun diezaioke, testuan horrela jarraitzea "probableena" bada.

2. fasea — Fine-tuning gainbegiratua (SFT: instrukzioak jarraitzen ikasi). Modeloari milaka adibide erakusten zaizkio "eskaera → erantzun on" bikote gisa, gizakiek (edo beste modelo batzuek) prestatuta. Hemen ikasten du elkarrizketa-formatua, aginduak betetzea, kodea eskatzen zaionean kodea ematea. *Analogia: irakaslearekin klaseak — adibide onak ikusi eta imitatzen ikasi*. Agenteentzat funtsezkoa: fase honetan **agente-trazak** ere erakusten zaizkio (tresnak deitzen dituzten elkarrizketa osoak), formatua ikas dezan.

3. fasea — Errefortzu bidezko alineazioa (RLHF: zer nahiago dugu ikasi). Gizakiek modeloaren bi erantzun konparatzen dituzte ("hau hobea da hura baino") eta modelo doitzen da nahiago ditugun erantzunak emateko: argiagoak, seguruagoak, erabilgarriagoak. *Analogia: tutorearen feedback-a — ez dizu erantzun berririk erakusten, baizik zureak fintzen laguntzen dizu.*

4. fasea — Errefortzu-ikaskuntza zeregin egiaztagarriekin (agenteen gakoa). Hau da fase berriena eta agenteentzat erabakigarriena. Modeloari **zeregin osoak** ematen zaizkio — programa bat idatzi testak pasatu arte, matematika-problema bat ebatzi, informazioa bilatu eta erantzun zuzena eman, tresna-sekuentzia bat erabili helburu bat lortzeko — eta **emaitza egiaztagarriaren arabera saritzen da**: testak pasatzen badira, saria; erantzuna zuzena bada, saria. Modeloak milioika saiakera egiten ditu eta ikasten du **zer estrategiek funtzionatzen duten benetan**. *Analogia: enpresako praktikak — hor ez zaituzte esaldi politengatik ebaluatzen, lana bukatuta dagoen ala ez baizik.*



36. Irudia: LLM modernoak lau entrenamendu-faseak eskola-ibilbidearen analogiarekin — liburutegia (pretraining), klaseak (SFT), tutorearen feedback-a (RLHF) eta enpresako praktikak (RL zeregin egiaztagarriekin).

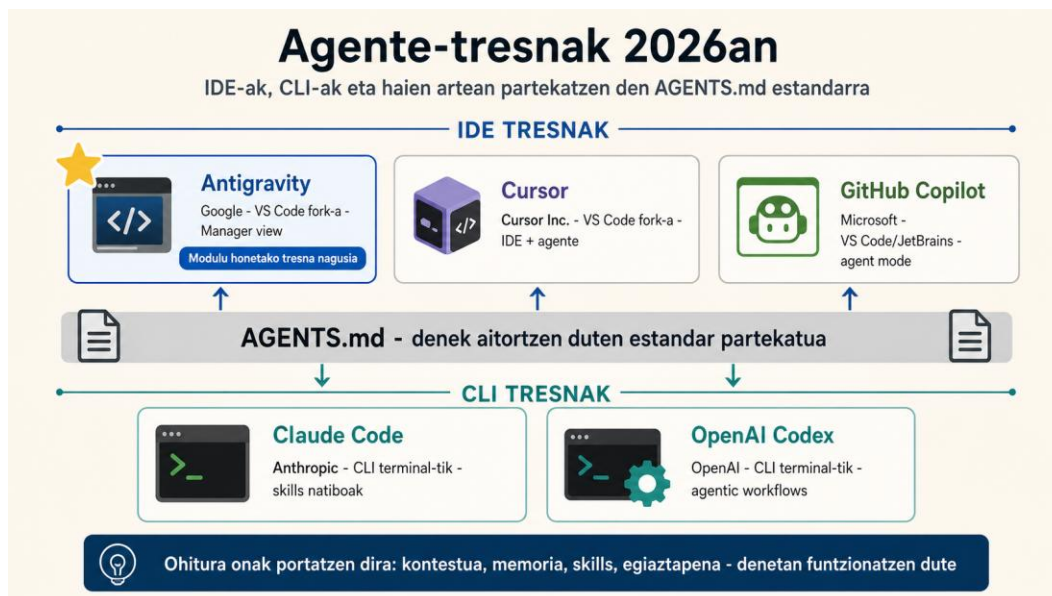
! Ideia nagusia Agenteek gero eta hobeto funtzionatzen dute ez modeloak "hitz-iragarle hobeak" direlako, baizik eta **4. fasean zuzenean entrenatzen direlako agente-lanetan**: planifikatu, tresnak deitu, errorea ikusi, zuzendu, berriz saiatu — bukle hori bera milioika aldiz praktikatu dute saria lortu arte. Antigravity-n agente batek zure testak exekutatu eta hutsegitea ikustean kodea berridazten duenean, ez da "kasualitatez asmatzen" ari: portaera hori bera entrenatu zaio. Horregatik dira gaur egungo "hausnarketa-modeloak" (Gemini 3 Pro, Claude-ren pentsamendu hedatua) hain onak lan konplexuetan: pentsatzeko denbora hartzea ere 4. fasean ikasi dute.

□ **Hausnartzeko** 1. fasea bakarrik balego (hurrengo hitza iragarri), zer gertatuko litzateke agente bati "konpondu bug hau" eskatzean? Pentsatu: testu "probableena" emango luke — kode itxura ona duena — baina inork ez lioke inoiz irakatsi testak pasatu behar direla. Zergatik aldatzen du dena emaitza egiaztagarriaren arabera sariak?

AGENTE-TRESNEN PANORAMA 2026AN

Modulu honetan **Antigravity** erabiltzen dugu (Google lizentzia + Manager view + integrazio osoa ikastetxean), baina lan-merkatuan beste tresna asko topatuko dituzu eta horiek menderatzeak balio handia du:

Tresna	Hornitzailea	Mota	Indargunea
Antigravity 2.0	Google	IDE (VS Code-ren fork-a)	Kudeaketa agentiko duala, Gemini 3 lehenetsi, modulu honetako tresna nagusia
Claude Code	Anthropic	CLI terminal-tik	Linux/devops fluxuetan oso indartsua, autonomia handia, skills sistema natiboa, Anthropic API-aren ekosistema
OpenAI Codex	OpenAI	CLI terminal-tik	ChatGPT-ren ekosistemarekin integratua, agentic workflows
Cursor	Cursor Inc.	IDE (VS Code-ren fork-a)	Tab autocomplete bikaina, IDE + agente hibridoa
GitHub Copilot	Microsoft / GitHub	VS Code / JetBrains luzapena	Industriaren ohiko aukera, "agent mode" berriena Microsoft-en ekosisteman



32. Irudia: Egungo agente-tresnen panorama — IDE-ak (Antigravity, Cursor) eta CLI-ak (Claude Code, OpenAI Codex), eta horien artean partekatzen den `AGENTS.md` estandarra.

□ **Albiste ona: ohiturak portatzen dira tresnen artean** Atal honetan ikasiko dituzun **sei ohitura nagusiek** (kontestua, memoria, skills, prompt engineering, egiaztapena, vibe coding-arekiko zuhurtasuna) **tresna guztietan funtzionatzen dute**. Bihar lan-postu

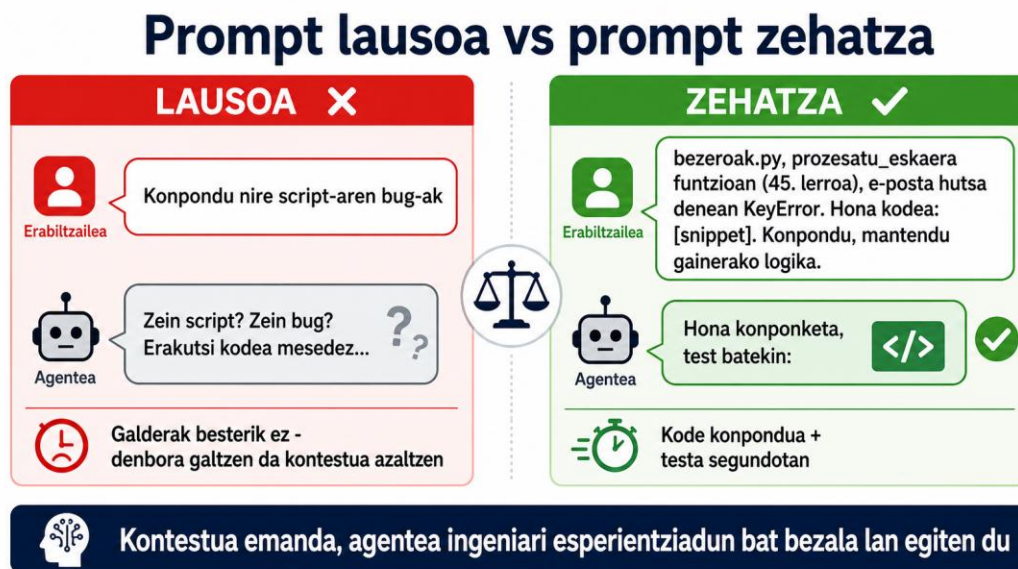
batek Claude Code edo Codex eskatuko balizu, ez zinateke berriz hasiko: interfaze ezberdina ikasi, baina **printzipioak berdinak dira**. Are gehiago, **AGENTS.md** fitxategia (§2.5.2-an sakon ikusiko duguna) **estandar partekatua** bihurtu da 2026rako: Antigravity, Claude Code, OpenAI Codex, Cursor eta beste batzuek **fitxategi bera** irakurtzen dute. Behin idatzita, edozein tresna-aldaketa garestirik gabe.

⚠ **Kontuz, alabaina, tresna-aldaketak benetan dira garestiak hizkuntza- eta integrazio-aldeak** "Ohiturak portatzen dira" esan dugu, baina ez nahastu: tresna-aldaketak benetako kostua dauka azpiegituran, integrazioan eta lankidetzan. Ez aldatu enpresa-tresna behin eta berriz "agente berriagoa atera delako" — onuren eta kostuen balantze argia egin. Eta beti egiaztatu zer datu pasatzen duen tresna bakoitzak hodeira (§5.7 etika eta segurtasuna ikusiko dugu sakon).

2.5.1 KONTESTUA EMAN: PROMPT ARGIA VS PROMPT LAUSOA

AA agenteekin lan egiten dakitenen eta ez dakitenen arteko diferentzia handiena **kontestuan** dago. Prompt lausoek erantzun lausoak ematen dituzte; prompt zehatzek, aldiz, kode funtzionala eta egokitua.

Agentea ez da telepata: ez daki zure proiektua zer den, zer fitxategitan zauden, zer Python bertsio erabiltzen duzun, ezta zer arazo zehatza topatu duzun ere. Hori dena **zuk eman behar diozu prompt-ean**. Antigravity-k erraztu egiten dizu kontestua eskaintzea: fitxategiak arrastatu daitezke chat-era, @ ikurraz aipatu fitxategi edo funtzio zehatzak, eta agente-alboko barrak proiektuaren egitura ikusten du.



27. Irudia: Bi prompt-en konparaketa zuzena — ezkerrean lausoa, eskuinean zehatza — eta agentearen erantzun ezberdinak.

Q **Adibidea: Mariak agenteari laguntza eskatzen dio** Mariaren LH-ko ikasle batek bere bezero-arretako script bat hobetzeko Antigravity-ren agentea erabili nahi du.

- **Prompt lausoa:** "Konpondu nire script-aren bug-ak". Agenteak galderak egiten dizkio: zein script? Zein bug? Mariak azaldu behar dio dena hutsetik.

- **Prompt zehatza:** "Nire `bezeroak.py` script-an, `prozesatu_eskaera` funtzioan (45. lerroa), bezeroaren e-posta hutsa denean `KeyError` jasotzen dut. Hona hemen funtzioa eta sarrera-adibide bat: [kodea]. Konpondu hori, baina mantendu gainerako logika eta gehitu test bat hutsa kasua estaltzen duena." Bigarrenak kode konpondua, testarekin eta azalpenarekin itzultzen du segundo gutxitan.

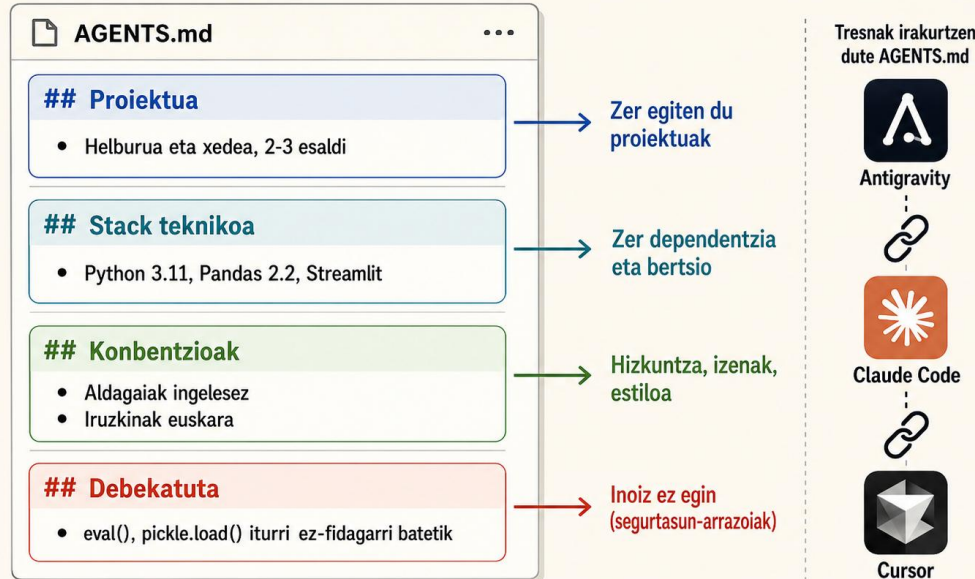
💡 **Idea nagusia** Prompt onak hau du: **zer arazo, non (fitxategia/lerroa), zer espero, zerk ez du funtzionatzen, adibideak/datuak**. Bost galdera horiek erantzunda, agenteak garatzaile esperientziadun bat bezala lan egiten du.

2.5.2 MEMORIA: AGENTS.MD, GEMINI.MD ETA CLAUDE.MD FITXATEGAAK

AA agenteak **memoria mugatua** du. Elkarrizketa baten hasieran azaldu zenuen guztia poliki ahaztuko zaio testuinguru-leiho bat betetzen den heinean, eta proiektu berri batera itzultzean dena hutsetik azaldu beharko duzu. Hori ezin da eskalatu.

Soluzioa: **kontestu-fitxategiak**. 2024-2025ean tresna bakoitzak bere formatuari eutsi zion (Claude Code-ren `CLAUDE.md`, Cursor-en `.cursorrules`, Aider-en `.aider.conf.yml`, eta abar), eta hori desordenatua zen — proiektu bakoitzean lau fitxategi paralelo idatzi behar zenituen. 2026rako, ordea, **AGENTS.md** estandar partekatua bilakatu da: **Antigravity, Claude Code (Anthropic), OpenAI Codex, Cursor eta Aider**-ek fitxategi bera sortzen dute. Behin ondo idatzita, ehunka elkarrizketatan denbora aurrezten du **edozein tresnatan**, eta tresna-aldaketak garestirik gabe egiten dira. (Claude Code-k oraindik `CLAUDE.md` ere onartzen du atzerako bateragarritasunagatik; AGENTS.md-ra biltzea gomendatzen da proiektu berrietan).

AGENTS.md - proiektuaren DNA didaktikoa agentearentzat



Behin idatzi, ehunka elkarrizketatan aurrezten du

28. Irudia: AGENTS.md fitxategi profesional baten anatomia — proiektu-deskripzioa, konbentzioak, debekuak eta hizkuntza-arauak.

Adibide praktiko bat:

```
# AGENTS.md - Salmenta-analisisa proiektua

## Proiektua

Hilabeteko salmenta-datuen analisisa (Excel + CSV iturri).
Helburua: dashboard automatikoa Streamlit-en.

## Stack

- Python 3.11, venv
- Pandas, Plotly, Streamlit
- Datuak: `data/raw/` (ez aldatu, beti kopia egin)

## Konbentzioak

- Aldagai-izenak ingelesez
- Iruzkinak euskaraz
- Inoiz ez committatu `.env`

## Debekatuta

- `pickle.load` (segurtasun-arrisku)
- Inoiz ez aldatu `data/raw/` zuzenean
```

Honekin agenteak ez du behar gehiago galdetzea "zer pakete erabiltzen duzu?" edo "ingelesez ala euskaraz?". Hasieratik dakizki.

Q Adibidea: Ikastetxean ikasleak lankidetza-proiektu bat egiten ari dira. AGENTS.md sortzen dute lehen egunean, biltegira igo, eta lau ikasleek partekatzen dute. Edozeinek elkarrizketa berri bat irekitzean Antigraity-rekin, agenteak dakizki: euskara hutsean, scikit-learn aukeratu modeloentzat, Pandas datuetarako, eta ez aldatu `data/raw/`. Lau pertsonak, indibidualki promptak idatzi behar zituztenean orduak galtzen zituzten, orain minutuak.

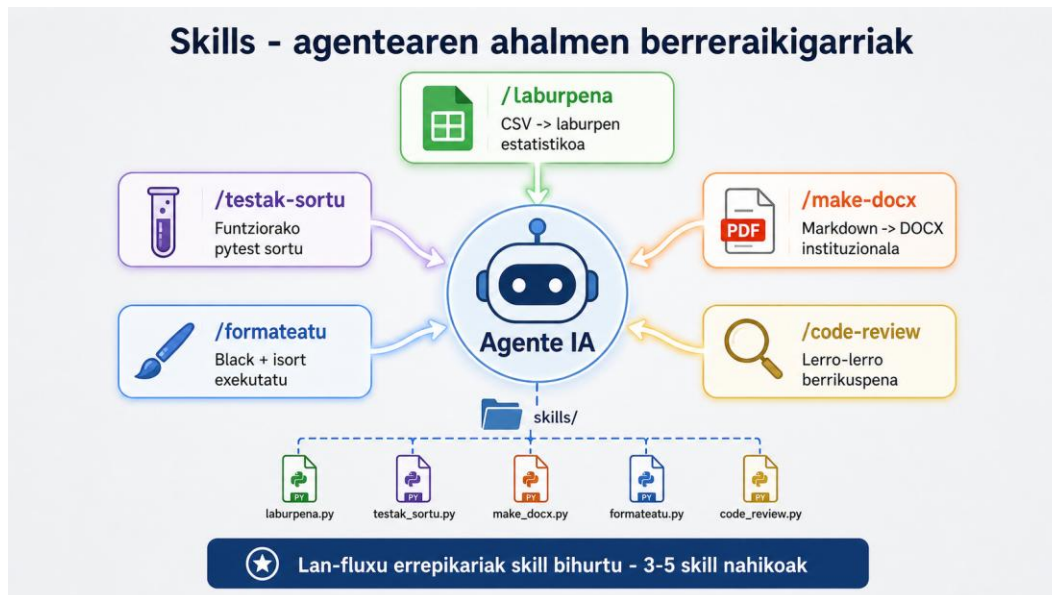
Idea nagusia AGENTS.md/GEMINI.md/CLAUDE.md proiektuaren "DNA didaktikoa" da agentearentzat. Behin ondo idatzita, ehunka elkarrizketatan denbora aurrezten du eta proiektu-koherentzia bermatzen du.

2.5.3 SKILLS ETA TRESNA PERTSONALIZATUAK

Skills (Claude Code-n) edo slash-komando pertsonalizatuak (Cursor, Antigraity) **agentearen ahalmen berrerabilgarriak** dira. Adibidez, beti egiten duzun "datu-multzo bat irakurri eta laburpen estatistiko bat sortu" lan-fluxua skill batean bilduta, hurrengoan idatzi beharrean `/laburpena datu.csv` exekuta dezakezu zuzenean.

Skill bakoitzak ditu:

- **Izena** (slash-komando bezala deitzeko)
- **Argibideak** (zer egin behar duen, pausoz pauso)
- **Onartutako sarrerak** (fitxategi-mota, parametroak)



29. Irudia: Skill bat nola gehitzen zaion agenteari ahalmen-modulu berrerabilgarri gisa.

□ **Adibidea:** *make-docx* skill bat Proiektu honetan Markdown-etik DOCX sortzeko skill bat egin dugu (*Skills/DOC_Template/*). Eduki bera, behin idatzi, ehun proiektutan berrerabili. Skill bat egiteak inbertsio bat eskatzen du, baina gero ordainduta dago.

△ **Kontuz** Ez sortu skill bat erabili ez duzun lan-fluxurako. Skill bakoitzak mantentzea eskatzen du, eta erabili gabe geratzen direnek **agentea nahasten dute** ("zergatik dauka 50 skill nire proiektuak?"). Sortu **3-5 erabilita dauden lan-fluxutarako** soilik.

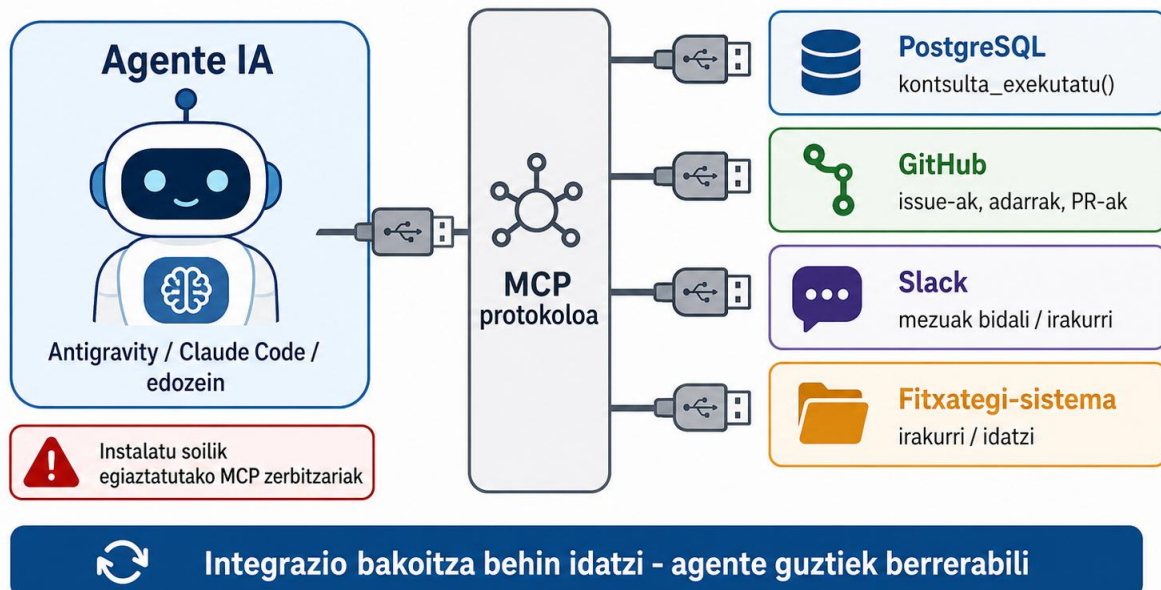
MCP: agentea kanpoko sistemetara konektatzeko estandarra

Skills-ek agenteari **jakintza** ematen diote (nola egin zerbait); baina agenteak askotan **kanpoko sistema batera konektatu** behar du: datu-base bat kontsultatu, GitHub-eko issue-ak irakurri, Slack-era mezu bat bidali. Horretarako estandarra **MCP (Model Context Protocol)** da: Anthropic-ek 2024an sortu zuen protokolo irekia, eta 2025erako industria osoak hartu zuen (OpenAI, Google, Microsoft). Antigravity-k, Claude Code-k, Cursor-ek eta agente-framework nagusiek MCP onartzen dute.

Funtzionamendua sinplea da: **MCP zerbitzari** batek sistema baten gaitasunak agerian jartzen ditu modu estandarrean (adib. "PostgreSQL zerbitzari honek *kontsulta_exekutatu* tresna eskaintzen du"), eta edozein agente MCP-bateragarrikeri erabil ditzake tresna horiek, integrazioz programatu gabe. USB-aren analogia erabiltzen da askotan: **MCP agenteen "USB ataka" da** — behin estandarra ezarrita, edozein gailu (sistema) konekta daiteke edozein ordenagailutara (agentera).

MCP - agenteen 'USB ataka'

Model Context Protocol: kanpoko sistemak modu estandarrean konektatu



35. Irudia: Agente bat MCP zerbitzarien bidez kanpoko sistemetara konektatuta — datu-basea, GitHub, Slack eta fitxategi-sistema, denak protokolo estandarra erabiliz.

Skills vs MCP — noiz zer:

Ezaugarria	Skill	MCP zerbitzaria
Zer ematen dio agenteari	Jakintza / errezeta (nola egin)	Sarbidea / konexioa (nora iritsi)
Adibidea	"/laburpena: CSV irakurri eta estatistikak sortu"	PostgreSQL-era konektatu eta kontsultak exekutatu
Nork sortzen du	Zuk (markdown + argibideak)	Komunitateak edo zuk (zerbitzari-programa)
Non bizi da	Zure proiektuan	Prozesu independente gisa (lokala edo urruna)

Q Adibidea: GitHub MCP zerbitzaria Zure agenteari "itxi ezazu #42 issue-a eta sortu adar berri bat konponketarako" eskatzen diozu. Agentea GitHub-eko **MCP zerbitzarira** konektatuta badago, zuzenean egiten du: issue-a irakurri, itxi, adarra sortu. MCP-rik gabe, zuk eskuz egin beharko zenuke edo agente bakoitzerako GitHub-integrazioa programatu. MCP zerbitzari ofizialak daude GitHub, Slack, PostgreSQL, Google Drive eta ehunka sistematarako.

⚠ Kontuz: MCP zerbitzariak ere egiaztatu ClawHub-eko skill maltzurren kasuak (§2.5.7-an ikusiko duguna) MCP zerbitzariekin ere gerta daitezke: hirugarren batek idatzitako MCP zerbitzari bat instalatzeak haren kodea zure makinan exekutatzea esan nahi du, eta zure agenteak hari pasatzen dizkion datuak ikus ditzake. **Instalatu soilik iturri ofizialetako edo egiaztatutako MCP zerbitzariak**, eta begiratu zer baimen eskatzen dituen.

2.5.4 PROMPT ENGINEERING MINIMOA

Prompt engineering hau ez da PhD bat behar duen ezagutza esoterikoa. **Bost printzipio sinplek** kalitatearen %80 dakarte:

- Argitasuna berritsukeriaren gaintetik** — 5 lerroko prompt zehatz batek 50 lerroko orokorrak baino emaitza hobea ematen du.
- Zehaztu formatua** — "JSON itzuli", "lista numeratu" edo "kode-bloke bakar bat". Bestela agenteak zer iruditzen zaion erabakitzen du.
- Few-shot adibideak** — lan ohikoa ez bada, eman 1-2 sarrera-irteera adibide. Agenteak azkar baten asmatuko du patroia.
- Scaffolding edo aldarnatzea, prompt luzeegien orde** — hasi prompt sinpleekin; arazoak agertu ahala gehitu xehetasunak. Ez idatzi 300-hitzeako prompt erraldoia hasieran.
- Lasaitasun-eskaera** — "Hartu minutu bat pentsatzeko, erantzun baino lehen" gisako esaldiek (Anthropic-ek behaketak ditu) erantzunaren kalitatea hobetzen dute zeregin konplexuetan.

Q Adibidea: Iteraziora ezarpena

- **Lehen iterazioa:** "Idatzi funtzio bat ariketa-emaitzak prozesatzeko" → agenteak suposizioa asko egin behar ditu.

- **Bigarren iterazioa** (lehen erantzunaren ondoren): "Hori ondo dago, baina nahi dut JSON itzuli, eta gehiketako batez bestekoa kalkulatu" → zehazten du.

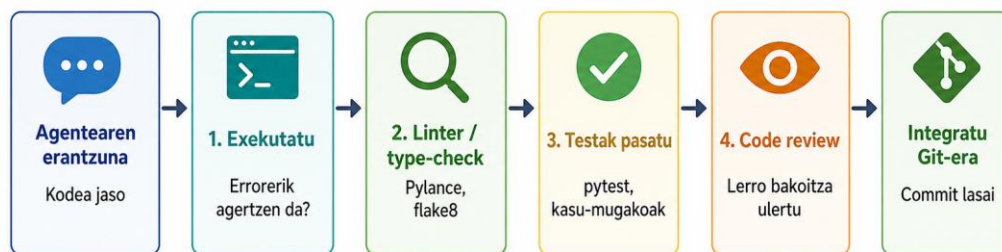
- **Hirugarren iterazioa**: "Gehitu test sortzaile bat hau probatzeko, pytest-ekin" →osatzen du. Hiru iterazio txikiek emaitza hobea ematen dute prompt erraldoi bat baino.

! **Idea nagusia** Aurrena probatu prompt sinplea. Bikain funtzionatzen badu! Ordea, funtzionatzen ez badu, gehitu testuingurua eta adibideak — ez idatzi 200-hitzeko prompt erraldoia hasieran inolako frogarik gabe.

2.5.5 EGIAZTAPENA: TRUST BUT VERIFY

Hau da ohitura **kritikoenetakoa**, eta gehien hausten den behi sakratua. AA agenteak haluzinatzen du: API-dei desberdinak asmatzen ditu, existitzen ez diren liburutegiak deitzen ditu, edo logika plausible-baina-okerra proposatzen du. Eta dena **konfiantzaz osoarekin** itzuli ohi du.

AI-kodearen egiaztapen-fluxua: lau urrats



Urrats bat saltatzen baduzu, arrisku ezkutua zure proiektuan



Trust but verify - agenteak alukinatzen du sintaxia eta API-ak

30. Irudia: AI-kodearen egiaztapen-fluxua — agentearen erantzuna → exekutatu → testak pasatu → linter → code review → integratu.

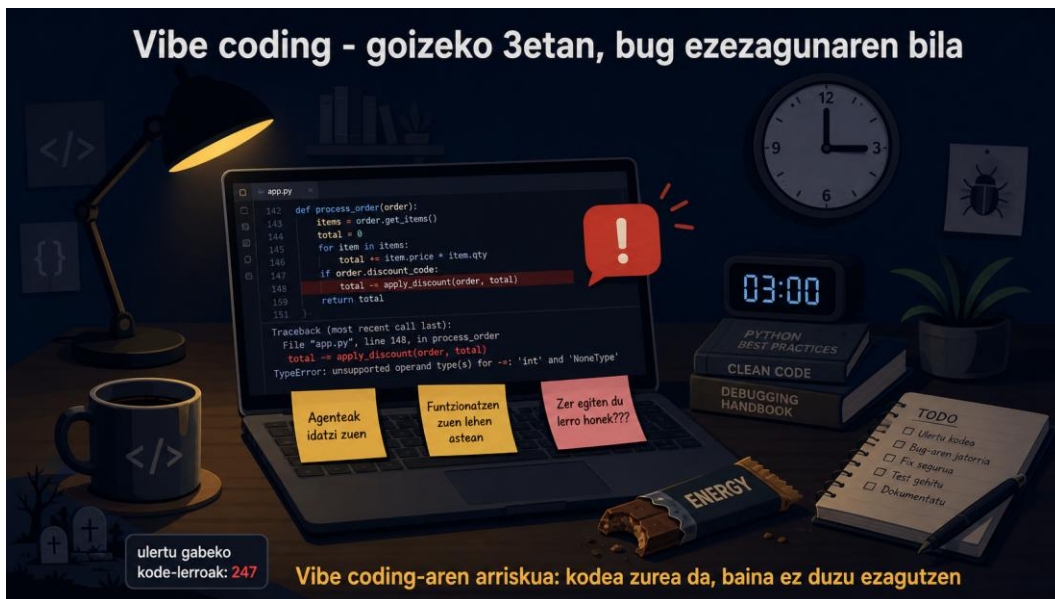
Egiaztatzeko **lau urrats** beti:

- Exekutatu**: ez kopia eta sinetsi. Agenteak `df.fillna(method='forward')` proposatu badizu, **exekutatu**. Pandas 2.0-tik metodo hori kenduta dago — agenteak ez du jakin. Errorrea agertzean ikusiko duzu.
- Linter / type-check**: Pylance-k ez badu errorerik markatzen, sintaxia zuzena da gutxienez.
- Testak**: agenteak berak testak pasatzeko kodea eraiki eta exekutagarri baldin badituzu birpasatzeko, hainbat hobe.
- Code review**: irakurri kodea lerroz lerro **eta ulertu zer egiten duen**. Ez bada ulertzen, ez onartu. "Ondo dirudi" ez da nahikoa. Azalpenak eskatu dakizkiokezu atalen batean.

□ **Hausnartzeko** Zenbat aldiz zuzenean onartu duzu agenteak proposatutako kodea? Eta zenbat aldiz ondo aztertu duzu? Zer pasako litzateke kode horretan SQL injection bat edo hardcoded API gako bat agertuko balitz?

2.5.6 "VIBE CODING"-AREN ARRISKUAK

"Vibe coding" lan egiteko modu berriaren izena da: programatzaileak ez du kodea idazten, agenteari "vibe" edo *bibrazio/intuizio/sentipen* bat eskaintzen dio ("egidan/eginzak halako gauza bat ondo burutuko duen aplikazio bat") eta agenteak idazten du. Tentagarria da emaitza azkarrek dakartzalako, baina **arritsu ezkutua** ditu — horiek identifikatzen jakiteak egiten dute LH-teknikari ona maisu.



31. Irudia: Programatzaile bat goizeko 3etan, agenteak idatzitako kodean ezagutzen ez duen bug bat aurkitzen — ulertzen ez duen kode-lerroa.

Arrisku nagusiak:

- **Mantentzen ez den kodea:** kodea idatzi gabe, baina mantendu egin behar duzu (3 hilabete geroago). Bug bat agertzean ez duzu jakingo non begiratu.
- **Segurtasun-arazoak:** agenteak ez ditu beti praktika onak ezagutzen; hardcoded passwords, SQL injection, dependentzia ahulak.
- **Eredu-mendekotasuna:** agentearen ekosistemaren menpe zaude. Gemini API geldituz gero, ez dakizu lan egiten.
- **Trebetasun-galera:** ez baduzu inoiz `dict` baten metodoak buruz erabiltzen, eta agentea behar duzu lerro bakoitzeko... lan-merkatuan zer egingo duzu internetetik gabeko elkarriketa teknikoan edo sarbide-frogan?

⚠ **Kontuz: Heppner kasu erreala** 2026ko otsailean, enpresa batek bere produkzio-kodea galdu zuen sintaxi zaharkitu bat erabiltzeagatik — AA agentea batek proposatua, ikasleak ulertu gabe onartua, eta hiru hilabetez produkzioan inork detektatu gabe egon zen. Bezzeroaren datuak ondo prozesatzen ari ez ziren, eta ikuskaritza bat behar izan zen

detektatzeko. Lezioa: agenteak ez ditu gauza guztiak ezagutzen — bere entrenamenduetan oinarritzen da, eta horiek **zaharkituak edo desagokiak** izan daitezke.

□ **Hausnartzeko** Sortu duzun azken proiektuari begira: zenbat lerroan dakizu zehazki zer egiten duten? Eta zenbat dira copy-pasted agentearen erantzunetatik bertan begiratu gabe? Hori da zure proiektuko arrisku-portzentajea.

2.5.7 AGENTE AUTONOMOAK ETA SEGURTASUNA: OPENCLAW, NEMOCLAW ETA NON EZ INSTALATU

Atal honetan ikusi ditugun ohiturak agentea **garapen-tresna** gisa erabiltzearen inguruan dira (Antigravity, Claude Code, Codex). Baina **agente autonomoen** familia bat ere existitzen da: zure makinan instalatzen direnak eta AA-eredu batek esandakoa **zure baimen propiorik gabe** automatikoki exekutatzen dutenak — emailak bidaltzen, fitxategiak aldatzen, web-eskaerak egiten. **OpenClaw** da 2026an horren adibide paradigmaticoa, eta bere historia ezagutzeak garrantzi handia du FP-tekniko batentzat.

OpenClaw Peter Steinberger-ek sortutako kode irekiko agente autonomoa da. Aste gutxitan GitHub-en **135.000 izar baino gehiago** bildu zituen, eta `curl | bash` lerro bakar batekin instalatzen da. Modelo desberdinekin lan egiten du (Claude, GPT, lokala) eta zure makinaren **sarbide osoa** hartzen du. Dokumentatutako arazoak (Reco, Barracuda, NSFOCUS, IBM X-Force, 2026):

- **335 skill maltzur** banatu ziren ClawHub-en (haien marketplace publikoan), komunitateak detektatu baino lehen.
- **ClawJacked**: web orri batek isilik bahitu dezake zure agente lokala WebSocket-aren ahultasun bidez.
- **npm hornidura-katea**: pakete npm konprometitu batek garatzaileen ordenagailuetan OpenClaw isilik instalatu zuen.
- **iMessage 500 mezu**: erabiltzaile batek 500 mezu baino gehiago bidaltzen zituen agente bat aurkitu zuen, bere kontaktuei eskaerarik gabe.
- **Aseguru-disputa autonomoa**: agente batek bere kabuz aseguru-konpainia bati erreklamazio bat hasi zion.

NemoClaw NVIDIA-ren erantzun ofiziala da: kode irekiko segurtasun-pila bat OpenClaw-rentzat, lau geruzaz osatua (orkestrazioa, **OpenShell** kernel-mailako sandbox, **Nemotron 3 Super 120B** modelo lokala, Privacy Router). Komando bakar batekin instalatzen da OpenClaw-en gainean. Baina **NVIDIA-k berak onartzen ditu mugak**:

"While OpenShell provides robust isolation, remember that no sandbox offers complete protection against advanced prompt injection." — NVIDIA Technical Blog

OpenClaw nora bai eta nora ez

Agente autonomoa instalatzeko aukerak - datuen arriskuaren arabera

✗ **Portatil pertsonala**

- API-gakoak, kredentzialak
- Datu pertsonalak, kontaktuak
- Sare pertsonala

EZ INSTALATU

! **VPS / VM / Devcontainer**

- GitHub Codespaces, Hetzner VPS
- VirtualBox snapshot-arekin
- Proiektu-mailan isolatuta

ONARGARRIA - kostu/onura ona

✓ **Ekipo isolatua / NemoClaw**

- Ikastetxeko ekipo dedikatua
- NemoClaw + GPU enterprise
- Sandbox + audit logging

GOMENDATUA - ideala ikaskuntzan eta ekoizpenean

! '5 minutuko froga bat' ez dago halakorik - agente autonomoek datu-fuga betikoak utz ditzakete

33. Irudia: Hiru testuinguruko hedapen-matrize bisuala — portatil pertsonalean **X**arriskua handia; VPS, makina birtuala edo devcontainer-rean ☐ aukera onargarria; ikastetxeko ekipo isolatua edo NemoClaw + GPU ☒ aukera onenak.

Non instalatu (eta non ez) OpenClaw edo antzeko agente autonomoa:

Testuingurua	Bertan?	Zergatik
Portatil pertsonala	☐ EZ	.env-fitxategiak, kredentzialak, datu pertsonalak arriskuan
GitHub Codespaces / Devcontainer	☐ Bai	Edukiontzia-mailan isolatuta, proiektuaren scope-an
Makina birtuala (VirtualBox, UTM, Multipass)	☐ Onargarria	Snapshot aurretik egin eta restore ondoren
VPS dedikatua (Hetzner CX22 ~5 €/hil, DigitalOcean)	☐ Bai	Zure ingurune pertsonaletik aparte
Ikastetxeko ekipo isolatua	☐ Idealena ikaskuntzan	Datu pertsonalik gabe, sare propioa
NemoClaw + GPU dedikatua (DGX, Vertex AI)	☐ Enterprise-rako	NVIDAA-ren irtenbide-arkitektura ekoizpenean

⚠ Kontuz: "5 minutuko froga bat" ez dago halakorik Tentazioa egon daiteke "froga azkar bat" eginez OpenClaw instalatzeko zure portatil pertsonalean "gero kentzen dut". Baina nahikoa da agentea behin abian jartzea zure .env, SSH-gakoak edo bezero-datuak filtratzen has dadin — eta hori ezin duzu atzera bota. **Ohitura on bat bizitza osorako da:** agente autonomoekin esperimentatu nahi baduzu, beti egin VM, edukiontzia, VPS edo ikastetxeko ekipo isolatu batean. Ez besterik.

💡 Ideia nagusia Bi agente-mota daude: **garapen-tresnak** (Antigravity, Claude Code, Codex) zure kodearekin lan egiten duten asistenteak — eta **agente autonomoak** (OpenClaw eta antzekoak) zure sistema osoarekin elkarrekarritzen dutenak. Lehenak egunero erabiltzeko dira; bigarrenak **inguru kontrolatuan bakarrik**. Edozein agentek zure sisteman komandoak exekutatzeko ahalmena duenean, isolamendua erabaki kritikoa da.

□ **Hausnartzeko** OpenClaw ez da bakarrik dagoen agente autonomoa: **Open Interpreter, Aider, gptme** eta beste hainbatek dute antzeko gaitasuna. Etorkizunean OpenClaw kolapsatzen bada bere arazoengatik, beste agente baliokide bat aterako da. Kontzeptua iraunkorragoa da produktua baino — eta ohiturak (non instalatu, zer isolatu, zer datu ez utzi eskura) gauza guztietan balio dute.

ATALAREN LABURPENA: SEI OHITURA NAGUSIAK

#	Ohitura	Erregela praktikoa
1	Testuingurua eman	"Fix this" ez. Eman fitxategia, lerroa, espero den portaera, adibideak
2	Memoria iraunkorra	AGENTS.md/CLAUDE.md proiektu bakoitzean, biltegira igota
3	Skills berrerabilgarriak	3-5 lan-fluxu errepikariak skill bihurtu — gehiago ez
4	Prompt sinpletik konplexura	Hasi laburrarekin; aldarnatzea, saihestu prompt luzeak
5	Beti egiaztatu	Exekutatu + linter + testak + code review (lau urratsak)
6	Vibe coding-i izan beldur	Ulertu duzun lerro bakoitza onartu — bestela ezagutza ez da zurea

□ **Modulu honetako ohitura iraunkorrak** Sei ohitura hauek ez dira "atal honetarako"; **modulu osoaren ohiturak** dira. 2., 3., 4. eta 5. gaietan agenteari deituko diozu Pandas, scikit-learn, FastAPI, Docker eta agente korporatiboekin laguntzeko. Ohitura hauek aplikatuz, ikasturte amaieran benetan programatzen ikasiko duzu — ezazu besarkatuitsu-itsuan "vibe coding"!

□ **Ariketa 2.5 · 30 min · banaka Egoera:** Antigraity 2.0 (IDE eta aplikazio orkestratzailea) instalatu dituzu (Ariketa 2.1) eta zure lehen Python proiektua hasi nahi duzu modulu honetarako.

Eginkizuna — Hiru zatitan:

(a) AGENTS.md sortzea — Sortu `AGENTS.md` fitxategi bat zure proiektuaren karpetan. 8-12 lerro izan ditzala, hauek aipatuz: proiektuaren xedea, Python bertsioa, hizkuntza-arauak (euskara/gaztelania/ingeleza non), zerbait debekatua (adib. `eval()`, `pickle.load`). Gehitu Git-era.

(b) Prompt-konparaketa — Eskatu agenteari Python funtzio bat (CSV bat irakurri eta laburpen estatistikoa itzuli). Bi promptak idatzi:

1. **Lausoa:** 10 hitz inguru ("Idatzi CSV irakurtzen duen funtzioa.")
2. **Zehatza:** 60+ hitz, kontestuekin (fitxategi izena, zutabeak, espero den emaitza JSON formatuan, errore-kudeaketa).

Konparatu bi erantzunak: zenbat lerro? Errore-kudeaketarik? Tipoak markatuta?

(c) Egiaztapen-zikloa — Hartu (b)-ko bigarren erantzuneko kodea eta **inolako moldaketarik gabe** exekutatu. Funtzionatu al du? Linter-ak (Pylance/flake8) zer dio? Idatzi 100-hitzeako hausnarketa: zer aldatuko zenuke ekoizpenean erabili aurretik?

(d) Agente autonomoaren isolamendua — Imajina ezazu OpenClaw probatzeko gogoia duzula (§2.5.7-an azaldua). Hautatu non instalatuko zenuke zure portatil pertsonala saihestuz, eta justifikatu hautaketa 3 lerrotan: zer aukera (VPS, devcontainer, VM, ikastetxeko ekipoa), zer datu pertsonal duzu babesten, eta zer kostua/komplexutasun erlazioa. **Eztabaidatzeko taldean:** Zer ohitura aplikatu duzu Ariketa 2.5 hau egitean? Modulu osoaren gainerako gaietan zer ohitura mantenduko duzu agentea erabiltzean?

3. Python Aurreratua Datuetarako

Python-en indarra ez datza soilik sintaxi sinplean, baizik eta kode idazteko modu **pythoniko** batean. Atal honetan datuak modu adierazkor eta eraginkorrean tratatzeko hiru teknika-multzo nagusi ikasiko dira: **list comprehensions** (zerrendak, hiztegiak eta multzoak sintaxi trinkoan), **funtzio anonimoak eta tresnak funtzionalak** (`lambda`, `map`, `filter`, `reduce`), eta **datuen iraupena** (JSON, YAML eta Pickle formatuak).

3.1 List Comprehensions

List comprehension-ak Python-en zerrenda berriak modu labur eta adierazkorrean sortzeko bide nagusia dira. Matematikako multzoen notazioan inspiratuta daude ($\{x^2 : x \in \mathbb{N}, x < 10\}$), eta horrek azaltzen du zergatik diren hain naturalak datuak transformatzeko.

```
# Forma klasikoa: for loop
karratuak = []
for n in range(10):
    karratuak.append(n ** 2)

# Baliokidea list comprehension bidez
karratuak = [n ** 2 for n in range(10)]

print(karratuak)
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Sintaxi orokorra:

```
emaitza = [espresioa for elementua in iterable if baldintza]
#          ^^^^^^^^^      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^      ^^^^^^^^^^^^^
#          zer egin      nondik hartu          filtroa (aukerazkoa)
```

List comprehension vs For loop

For loop tradizionala

```
karratuak = []
for n in range(10):
    karratuak.append(n ** 2)
```

4 lerro

Errendimendua:
oinarrizkoa

List comprehension

```
karratuak = [n ** 2
              for n in range(10)]
```

1 lerro

Errendimendua:
% 20-40 azkarragoa

Emitza berdina, bide ezberdinak

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

11. Irudia: List comprehension eta for loop tradizionalaren konparaketa bisuala — eraketa, irakurterraztasuna eta errendimendua.

BALDINTZEKIN

```
# Bikoitiak soilik
bikoitiak = [n for n in range(20) if n % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Zerrendako datu-garbiketa
datuak_gordinak = [" Ane ", "MIKEL", " leire ", "JON"]
datuak_garbi = [d.strip().capitalize() for d in datuak_gordinak]
# ['Ane', 'Mikel', 'Leire', 'Jon']
```

DICTIONARY ETA SET COMPREHENSIONS

```
# Dictionary comprehension
ikasleak = ["Ane", "Mikel", "Leire"]
notak = {ikaslea: 0 for ikaslea in ikasleak}
# {'Ane': 0, 'Mikel': 0, 'Leire': 0}
```

```
# Set comprehension (bikoiztuak kendu)
hitzak = ["kaixo", "mundua", "kaixo", "python", "mundua"]
bakanak = {h for h in hitzak}
# {'kaixo', 'mundua', 'python'}
```

Q Adibidea: Datu-garriketa eskema baten parte Bezeroaren e-postak jaso dituzu:

`["ane@firma.eus", "MIKEL@FIRMA.EUS", "leire@firma.eus"]`. Datu-basera sartzeko: garbitu zuriguneak, jarri minuskulan, eta iragazi bakarrik baliodunak (puntu eta @ dituztenak). Lerro bakarrean: `garbi = [e.strip().lower() for e in epostak if "@" in e and "." in e]`. List comprehension bat = jada erabakitako garriketa-arau bat.

! Ideia nagusia List comprehensions Python-en pythonikoak diren patroï nagusietakoak dira: helburua deskribatzen dute, ez urrats-urratseko prozedura. Errazago irakurtzen dira, normalean azkarragoak dira, eta talde-lanean argitasuna hobetzen dute.

Δ Kontuz: konplexutasun-muga Irakurgarritasuna lehentasuna izan behar du beti. Comprehension batek lerro batean **bi for habiaratu eta hiru baldintza** sartzen baditu, jadanik ez da "trinkoa eta argia", "trinkoa eta ulertezina" baizik. Arau praktikoa: comprehension bat begi-kolpe batean ulertzen ez bada, hobe da **for** begizta tradizional bat erabiltzea.

□ **Ariketa 3.1 · 15 min · banaka Egoera:** Klasekideen zerrenda bat duzu hiztegi formatuko lerroekin (hiztegi-zerrenda): `ikasleak = [{"izena": "Ane", "nota": 7}, {"izena": "Mikel", "nota": 4}, {"izena": "Leire", "nota": 9}, ...]`.

Eginkizuna (list comprehensions erabiliz):

1. Lortu gainditu duten ikasleen izenen zerrenda (`nota >= 5`).
2. Sortu hiztegi berri bat ikaslearen izena → nota duena.
3. Lortu klasearen batez besteko nota.

3.2 Lambda, Map eta Filter

Programazio-paradigma desberdinak daude. **Inperatiboan**, ordenagailuari urrats zehatzak agintzen zaizkio. **Funtzionalean**, aldiz, transformazioak eta konposizioak deskribatzen dira. Python ez da erabat funtzionala, baina paradigma funtzionaleko tresna eskaintzen ditu: `lambda`, `map`, `filter`.

LAMBDA — FUNTZIO ANONIMOAK

```
# Funtzio arrunta
def bikoiztu(x):
    return x * 2

# Lambda baliokidea
bikoiztu = lambda x: x * 2
```

```
print(bikoiztu(5))    # 10
```

Erabilera tipikoak:

```
# 1. Ordenatzeko gako gisa
produktuak = [
    {"izena": "sagarra", "prezioa": 1.2},
    {"izena": "platanoa", "prezioa": 0.8},
]
ordenatua = sorted(produktuak, key=lambda p: p["prezioa"])
```

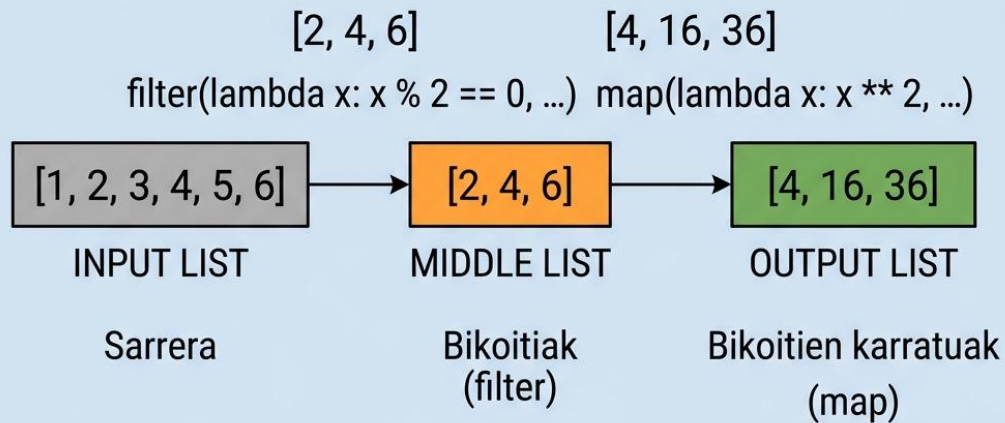
MAP ETA FILTER

```
# map: funtzioa aplikatu elementu bakoitzari
zenbakiak = [1, 2, 3, 4, 5]
hirukoak = list(map(lambda x: x * 3, zenbakiak))
# [3, 6, 9, 12, 15]

# list comprehension baliokidea: hirukoak = [x * 3 for x in zenbakiak]

# filter: elementuak iragazi baldintza batekin
bikoitiak = list(filter(lambda x: x % 2 == 0, range(10)))
# [0, 2, 4, 6, 8]
```

Map eta Filter — datu-pipeline funtzionala



Konposatu daitezke: edozein konplexutasunaren transformazio

12. Irudia: Map eta Filter funtzioen lan-fluxua bisualki — sarrera-zerrenda, funtzioa aplikatu, irteera-zerrenda iragazitakoa.

```
# datu-pipeline funtzionala
bikoitien_karratuak = list( #Flake8 formatuan 79 karaktere GEHIENEZ lerroko!
    map(lambda x: x ** 2,
        filter(lambda x: x % 2 == 0, range(10))
    )
)
print (bikoitien_karratuak)
# [0, 4, 16, 36, 64]
```

Q Adibidea: Eposta zerrenda filtratzea Marketin-talde batek 10.000 e-posta dauzka, eta `@firma.eus` domeinuko bezeroei bakarrik bidali nahi die kanpaina bat. `filter(lambda e: e.endswith("@firma.eus"), epostak)` lerro bakarrean filtratzen ditu, gainerako 9.500 jaurti gabe.

! **Idea nagusia** *map* eta *filter* askotan list comprehensions-ek ordezkatzeko ditzakete (ikus goiko adibidea) eta irakurterrazagoak izan daitezke. Lambda erabili: (a) sort-en eta GroupBy-en gako gisa, edo (b) funtzio txiki batek behin bakarrik erabili behar duenean.

□ **Ariketa 3.2 · 5 min · banaka Eginkizuna:** Idatzi dugu *lambda*, *map* eta *filter* erabiliz (pipeline konposatua): zerrenda bateko zenbaki bikoitien karratuaren kalkulua. Adibidea: $[1, 2, 3, 4, 5, 6] \rightarrow [4, 16, 36]$. Orain, idatzi ezazu berdina list comprehension erabiliz. Zein da irakurterrazagoa zuretzat?

3.3 Fitxategien Kudeaketa: JSON, YAML eta Pickle

Python-en datuak fitxategietatik irakurri eta idazteko hiru formatu nagusi erabiltzen ditugu: JSON, YAML eta Pickle. Bakoitzak bere kasu-erabilera dauka.

JSON

```
import json

# IDATZI: Python dict → JSON fitxategia
datuak = {
    "bezeroak": ["Ane", "Mikel"],
    "kopurua": 2
}
with open("datuak.json", "w") as f:
    json.dump(datuak, f, indent=2, ensure_ascii=False)

# IRAKURRI: JSON fitxategia → Python dict
with open("datuak.json", "r") as f:
    datuak = json.load(f)
print(datuak["bezeroak"]) # ['Ane', 'Mikel']
```

YAML

```
import yaml

# Konfigurazio-fitxategia
yaml_testua = """
proiektua: AI-AA
parametroak:
    learning_rate: 0.001
    batch_size: 32
"""
konfig = yaml.safe_load(yaml_testua)
print(konfig["parametroak"]["learning_rate"]) # 0.001
```

PICKLE

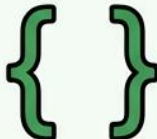
```
import pickle

# Python objektu konplexua serializatu eta gorde
```

```
eredua = {"pisuak": [0.5, -0.3, 0.8], "alborapenak": 0.1}
with open("eredua.pkl", "wb") as f:
    pickle.dump(eredua, f)

# Berreskuratu
with open("eredua.pkl", "rb") as f:
    eredu_berreskuratua = pickle.load(f)
```

Serializazio-formatuak: JSON, YAML eta Pickle

JSON	YAML	Pickle
		
<pre>{"izena": "Ane", "adina": 25}</pre>	<pre>izena: Ane adina: 25</pre>	<pre>b'\x80\x04\x95..'</pre>
Mota: Testua	Mota: Testua	Mota: Bitarra
Erabilera: APIak, weba	Erabilera: Konfigurazioak	Erabilera: Python objektuak (ML ereduak)
Segurua: Bai	Segurua: Bai (oharrak)	Segurua: Iturri ezagunekin bakarrik

Pickle: ez kargatu inoiz fitxategi ez-ezagunak

13. Irudia: JSON, YAML eta Pickle-en konparaketa bisuala — formatuak, irakurterraztasuna, segurtasuna eta erabilera-kasuak.

Formatua	Mota	Erabilera tipikoa
JSON	Testua	API, web, datu-trukea
YAML	Testua	Konfigurazioak (ML-parametroak, Docker, CI)
Pickle	Bitarra	Python objektu osoak (ereduak, datu-egiturak)

Q Adibidea: ML eredu bat gorde eta berreskuratu Entrenatutako scikit-learn eredu bat gorde nahi duzu, gerora produkzioan erabiltzeko. Pickle bidez (edo `joblib`-ekin, antzekoa baina hobea ereduentzat): `joblib.dump(eredua, "eredua.pkl")`. Web-zerbitzariak kargatzen du `eredua = joblib.load("eredua.pkl")` eta inferentzia egiten du. Eredu osoaren egitura konplexua serializatzen da byte-sortara, eta itzulera erabatekoa da.

⚠ Kontuz: Pickle segurtasun-arriskua Pickle-tik ez kargatu inoiz fitxategi ezezagunik! Pickle Python kodea exekutatzeke gai da deserializazioan, eta erasotzaileek hori erabiltzen dute kode maltzura sartzeko. Pickle erabiltzekotan, izan dadila **soilik zure fitxategi propioekin!**

💡 Ideia nagusia Format aukeraketa: testuinguruaren arabera erabakia da. JSON datu sinpleen etarako eta API etarako; YAML konfigurazio irakurterraza behar denean; Pickle izango da Python barneko objektu konplexu etarako.

□ **Ariketa 3.3 • 20 min • taldeka Egoera:** Hipermerkatu batek produktu-katalogo bat dauka (200 produktu, izena, prezioa, kategoria, stock).

Eginkizuna:

- Sortu hiztegi-zerrenda bat 5 produktu fikzioarekin.
- Gorde JSON gisa (`katalogoa.json`).
- Gorde YAML gisa (`katalogoa.yaml`).
- Berriz irakurri biak eta egiaztatu edukia berdina dela.
- Eztabaidatu: zein formatu hautatuko zenuke bezeroaren web-orriarentzat? Eta ML-entrenamendurako konfigurazio batentzat?

4. Bertsio Kontrola (Git)

Atal honetan **Git** ikasiko da, gaur egun bertsio-kontrolerako estandar absolutua. Edozein software-proiektu profesional Git-en oinarrituta dago. Ezagutza hau ezinbestekoa da edozein garatzaile-lanpostutan.

GIT GABE VS GIT-EKIN

GIT GABE: `script_v1.py`, `script_v2.py`, `script_v2_final.py`, `script_v2_finalFINAL.py`, `script_v3_marc.py`, `script_v3_marc_fix.py`... ezaguna egiten zaizu?

GIT-EKIN: fitxategi bakar bat (`script.py`); bere historia osoa erregistratuta (zer aldatu zen, noiz, nork, zergatik); edozein bertsio berreskuragarri minutu batean.



14. Irudia: Garapen-fluxuaren konparaketa — Git gabe (fitxategi-anarkia) vs Git-ekin (historia ordenatua, commit-en bidez).

GIT-EN ONURAK

- **Aldaketen historia:** nork, zer eta noiz aldatu zuen kode-lerro bakoitza erregistratuta dago.
- **Atzera itzultzea:** edozein aurreko bertsiora itzuli daiteke minutu batean.
- **Lankidetza:** hainbat pertsona aldi berean lanean, gatazkak modu ordenatuan ebaztea.
- **Esperimentuak:** branch-en bidez, ideia berriak proba daitezke arrisku gabe.

GIT-EN HIRU EGOERAK

Git-en fitxategiek hiru egoera nagusi izan ditzakete:

- **Working directory** — zure ordenagailuko fitxategiak, oraindik Git-i ez zaizkionak esleitu.
- **Staging area** — Git-i esleitu zaizkion fitxategiak, baina oraindik commit egin gabe (`git add`).
- **Repository** — commit egin diren fitxategiak, historian erregistratuta (`git commit`).

Q Adibidea: zenbat ordu galdu fitxategi-kudeaketa gabe Enpresakoko garatzaile batek bere ekipoarekin Git gabe lan egingo balu: bakoitzak bere fitxategi-bertsioak izango lituzke, e-postaz partekatuko lituzke, eta lankide batek bere aldaketak gainidatziko lituzke beste norbaiten lanaz ohartu gabe. Bisuala da: Git ezaguna ez bada, asteko 4-8 ordu gal daitezke fitxategi-kudeaketa nahasian. Git-ekin, 0 ordu.

💡 Ideia nagusia Git ez da "fitxategi-historian etiketak jartzeko sistema". Garatzaile-talde baten **lankidetza-protokoloa** da, software-ingeniariaren oinarritzko ohitura.

☐ **Hausnartzeko** Imajinatu enpresa batek 5 garatzaile dituela proiektu berean lanean. Git gabe, nola koordinatuko lirатеke? Zer arazo agertuko lirатеke? Probaketa-fasea nola gauzatuko zen?

☐ **Ariketa 4.1 • 10 min • banaka Eginkizuna:** Imajinatu enpresa berri batera lan egitera joan zarela eta ez dute Git erabiltzen. Idatzi paperean 3 arrazoi sendo zergatik Git instalatu beharko luketen. Aurkeztu arrazoiak ikaskideei.

HASIERAKO KONFIGURAZIOA (BEHIN BAKARRIK)

```
git config --global user.name "Zure Izena"
git config --global user.email "izena.abizena@uni.eus"
```

PROIEKTUA HASTEKO

```
# Aukera 1: Biltegia hasieratu (proiektu berria)
git init

# Aukera 2: Biltegia klonatu (dagoen proiektua)
git clone https://github.com/erabiltzailea/proiektua.git
```

OINARRIZKO FLUXUA

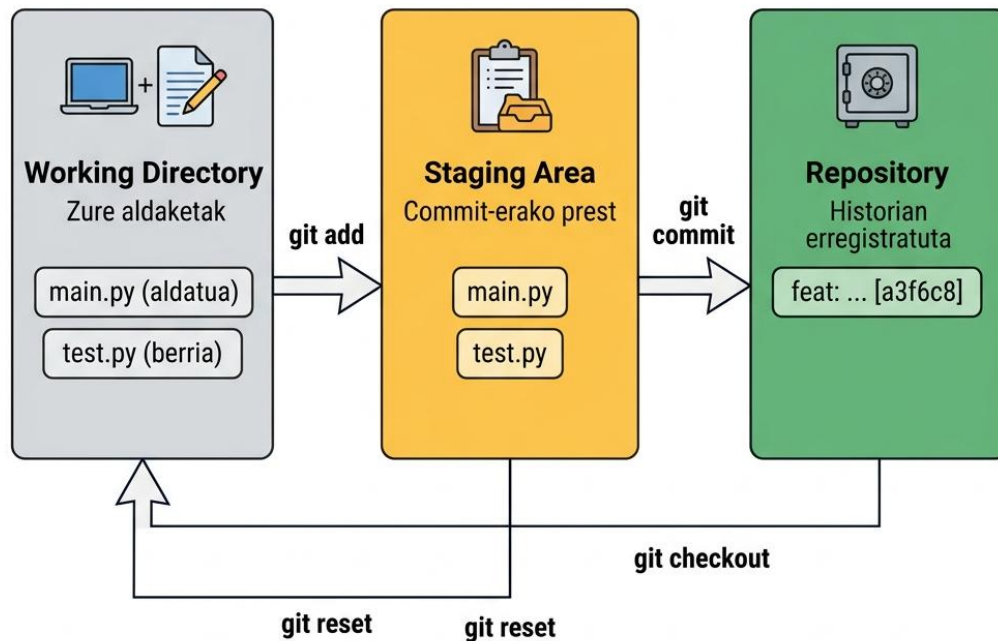
```
# 1. Egoera ikusi
git status

# 2. Fitxategiak Staging area-ra gehitu
git add fitxategia.py # Fitxategi bakar bat
git add .             # Dena

# 3. Commit sortu
git commit -m "feat: bezeroaren login funtzioa gehitu"

# 4. Historia ikusi
git log
git log --oneline --graph --all
```

Git-en hiru egoerak



15. Irudia: Git-en fitxategi-fluxua hiru egoeren artean — Working directory → Staging area → Repository, `git add` eta `git commit` komandoekin lotuta.

COMMIT-EN MEZUAK IDAZTEKO PRAKTIKA ONA

```
# TXARRA ✗
git commit -m "aldaketak"
git commit -m "gauza asko"

# ONA ✓ (Conventional Commits estandarra)
git commit -m "feat: bezero-modeloa sortu"
git commit -m "fix: login API errore-kode okerra konpondu"
git commit -m "docs: README eguneratu"
git commit -m "test: bezero CRUD testak gehitu"
```

Q Adibidea: Begi-kolpean historia ulertu Lankide bat zure proiektura sartzen da. *git log --oneline* egiten du eta ikusten du: "feat: bezero-modeloa sortu", "test: ..." "fix: ..." Hori irakurri eta jakiten du azken bi astetan zer egin den, zer bukatuta dagoen, eta zertan ari diren. Onuragarria denbora aurrezterako; ezinbestekoa onboarding-erako.

⚠ Kontuz: Commit-meziek **Conventional Commits estandarrari** jarraitu beharko liekete (*feat:*, *fix:*, *docs:*, *test:*).

💡 Ideia nagusia Commit mezuak ez dira norberarentzat: **ETORKIZUNeko ZURETZAT** idazten dira, edo lankide batentzat hiru hilabete geroago. Beti utzi commit-mezu argiak!

□ Ariketa 4.2 • 15 min • banaka

Eginkizuna: Hasi Git proiektu berria:

1. Sortu karpeta berria, *git init*.
2. Sortu *agur.py* agur sinplea inprimatzen duena.
3. *git add* eta *git commit -m "feat: agur funtzioa gehitu"*.
4. Aldatu *agur.py* agur pertsonalizatua egiteko (izena hartu).
5. Commit berria mezu egokiarekin.
6. *git log --oneline* egin eta egiaztatu bi commit ikusten dituzula.

4.3 Branching eta Merging

Branch-ak (adarrak) Git-en lan-kontzeptu nagusienetakoak dira. Aukera ematen dute ideia berriak *main*-ean ukitu gabe garatzeko, gero ondo dauden konfirmatzeko eta integratzeko.

```
# Branch berria sortu eta bertara aldatu
git checkout -b feature/ezaugarri-berria

# Branch-ak kudeatu
git branch
git branch -a
git checkout main

# Lokalak ikusi
# Lokalak + urrunekoak
# Beste branch batera aldatu (hemen, nagusira)

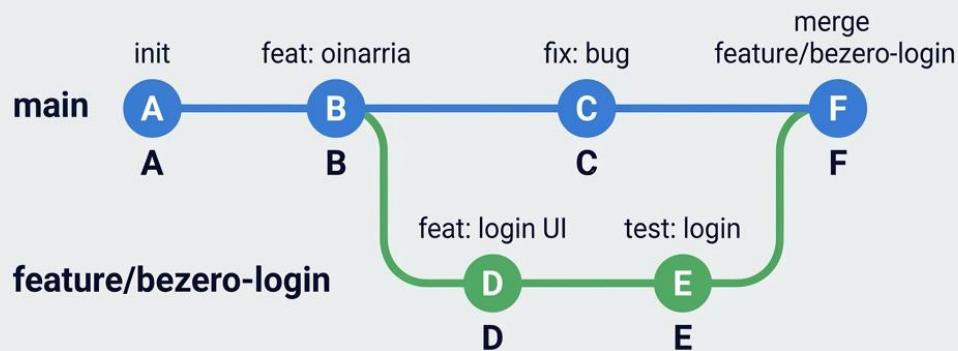
# Branch ezabatu
```

```
git branch -d feature/ezaugarri-zaharra
```

MERGING

```
# main-era itzuli eta branch-a batu
git checkout main
git merge feature/ezaugarri-berria # Gatazkarik (?) auskalo!
```

Git branch-ak eta merge



16. Irudia: Git-en branching-aren funtzionamendua — main branch eta feature branch-ak, merge-rekin amaitzen direnak.

GATAZKEN KONPONKETA (MERGE CONFLICTS)

Bi branch-ek kode-lerro bera aldatu badute, Git-ek ezin du automatikoki batu. Gatazka detektatzen du eta zuk eskuz konpondu behar duzu:

```
<<<<<<< HEAD
# Zure aldaketa
def kalkulatu(x, y):
    return x + y
```

```
=====
# Beste branch-aren aldaketa
def kalkulatu(x, y, z):
    return x + y + z
>>>>>> feature/ezaugarri-berria
```

Erabaki zein bertsio mantendu (edo nahaste bat), markatzaileak kendu (alabaina, gezi horiek fisikoki ezabatu), eta gero `git add + git commit`.

⚠ Oharra: era hori ez da lan-fluxu garbiena. Ordea, main adarra beti babestuta eta martxan mantentzea da praktika ona. Gatazka egongo baldin bada **lerro berdinak eguneratzearen mainean zein branchean**, konponketa eta probatu beharra ezin saihestuko da. **Bestela, automatikoki egingo luke fusioa**, esanda bezala. Eskua sartu beharko denetik, adar nagusian konponketak eta frogak hastea baino askoz hobe da adarrean egitea! Jariora: **ekarri bertsio eguneratua; fusionatu; konpondu lokalean; nagusian fusionatu.**

1. **aukera: adarrez aldatuz** adar lokaletik nagusira joan eta bertsio lokala zerbitzarikoarekin eguneratzera.
 - a. Ordenagailuko `main` adarra zerbitzarikoarekin eguneratu:


```
git checkout main
git pull origin main
```
 - b. Bere adarrera itzuli:


```
git checkout feature/ezaugarri-berria
```
 - c. `main`-eko aldaketak bere adarrean fusionatu:


```
git merge main
```
2. **aukera: adarrez aldatu gabe** eguneratu lokaleko `main` bertsioa eta ondoren merge egin harekin adar lokalean. Gatazka! Konpondu; berriz ere, `add` eta `commit` egin.
 - a. Zerbitzariko informazioa eguneratu:


```
git fetch origin
```
 - b. Zerbitzariko `main` adarra bere adarrean fusionatu:


```
git merge origin/main
```

Batera zein bestera eginda, leun leun fusionatuko dira eta frogatutako bertsioa partekatu da taldekideekin. Bigarrena, bizkorragoa; lehena, aldiz, tradizionalagoa/garbizaleagoa. Argi! "This branch has no conflicts" mezua irakurtzea merge egin aurretik, urre-bitxia-da!

Q Adibidea: Bi garatzaile, branch ezberdinak Anak `feature/bezero-login` branch-an lan egiten du, eta Mikelek `feature/eposta-bidalketa-n`. Lan biak independenteak dira eta egunero `main` branch-tik pull egiten dute eguneratuta egoteko. Astebururako ostiralean, Anak merge egiten du `main`-era. Mikelek aurrera segitzen du.

Astelehenean Mikelek merge egiten du **main**-era — Git-ek konpondu egiten du dena automatikoki (ez baitzituzten lerro berak ukitu). Lankidetzaz gatazka.

💡 **Idea nagusia** Branch-ek **isolazioa** ematen dute: zure ideia probatu dezakezu inori eragotzi gabe. Behin lan-emitza onargarria dela egiaztatuta, merge egiten duzu **main**-era.

⚠️ **Kontuz: main-ean ez landu zuzenean** Inoiz ez egin commitak zuzenean **main**-ean. Beti sortu branch berri bat ezaugarri (feature) edo matxura (bug-fix) bakoitzeko. Eta itzuli orduko, pull egin. Hori da praktika profesionalaren oinarria.

REMOTE KUDEAKETA

```
# Remote gehitu: lotu karpeta lokala Githubeko biltegi berriarekin
git remote add origin https://github.com/erabiltzailea/proiektua.git

# Aldatu adarraren izena "master"-etik "main"-era (Githubeko estandarra izateko)
git branch -M main

# Igo kodea lehenengo aldiz (eta ezarri adar lehenetsi gisa)
git push -u origin main          # Lehen push-a

# Push eta pull
git push                        # Ondorengo push-ak
git pull                        # Aldaketak jaitsi
```

PULL REQUEST (PR) FLUXUA

Pull Request-ek **kode-berrikuspen formala** eskaintzen dute. Workflow profesionalaren erdigunean daude: garatzaile batek bere lana proposatzen du PR gisa, beste kideek aztertzen dute, iruzkinak uzten dituzte, eta onartutakoan bakarrik integratzen da **main**-era.

Garrantzitsuena **PR baten deskribapen ona** da:

```
## Egindakoa
Erabiltzaile autentifikazio sistema gehitu da JWT bidez.

## Zergatik
Aplikazioak orain erabiltzaileen saio-hasiera eta sarbide-kontrola behar du.

## Nola probatu
1. `pip install -r requirements.txt`
2. `python -m pytest tests/test_auth.py`
3. Postman-en: POST /api/login

## Aldaketa nagusiak
- `auth.py`: JWT sortu eta egiaztatzeko funtzioak
- `routes/user.py`: Login eta logout endpoint-ak
- `tests/test_auth.py`: 12 test berri
```

Pull Request-aren bizi-zikloa



Kalitatea + ezagutza-transmisioa + erantzukizun-historia

17. Irudia: Pull Request-aren bizi-zikloa — branch sortu → commit-ak → PR ireki → berrikuspena → iruzkinak/aldaketak → onartuta → merge.

□ **Ariketa 4.3 · 25 min · taldeka Egoera:** Lankide batekin batera ariketa egingo duzue. Bietatik bat repo-aren jabea izango da, beste batek klonatuko du.

Eginkizuna:

1. Bietatik bat repo bat sortu GitHub-en, klonatu lokalean, fitxategi bat sortu.
2. Push egin GitHub-era.
3. Beste lankideak klonatu repo-a.
4. Bakoitzak bere branch berri bat sortu (*feature/A* eta *feature/B*).
5. Bakoitzak bere fitxategian aldaketak egin.
6. Bakoitzak push egin bere branch-ean.
7. GitHub-en bakoitzak Pull Request sortu.
8. Lankidearen PR-a berrikusi eta onartu.
9. Merge egin.

.GITIGNORE

`.gitignore` fitxategiak Git-i zein fitxategi eta karpeta **ez duen jarraitu behar** zehazteko balio du. Python proiektu baterako oinarrizkoak:

```
# Ingurune birtualak
.venv/
venv/

# Python cache
__pycache__/*
*.py[cod]
*.egg-info/

# Sekretuak
.env
*.env
credentials.json

# ML / Data
*.pkl
*.h5
data/raw/

# IDE / sistema
.vscode/settings.json
.idea/
.DS_Store

# Testing
.pytest_cache/
.coverage
```

Tresna lagungarria: gitignore.io— proiektuaren teknologiak idatzi eta `.gitignore` egokia sortzen du automatikoki.

PRAKTIKA ONAK

1. **Commit txikiak eta maizagoak**, mezu argiekin ([Conventional Commits](#)).
2. **Inoiz ez push egin zuzenean `main-era`** — beti pull request baten bidez.
3. **Branch estrategia argia** (Gitflow edo antzekoa): `main`, `develop`, `feature/*`, `bugfix/*`, `hotfix/*`.
4. **Sekretuen segurtasuna**: inoiz ez committatu `.env` edo API-gakoak. Igo bada, **giltza aldatu berehala**.

Q Adibidea: PR baten bizia Aritz ikaslea repo enpresarial batera sartu da praktketan. Bere lehen eginkizuna: bug bat konpondu loginean. Branch bat sortu (`bugfix/login-error`), kodea aldatu, commit egin "fix: login bezeroaren errore-kodea zuzendu", push egin, PR ireki. Senior batek berrikusi du, iruzkinak utzi ditu ("kontuz, hau ere aldatu beharko

zenuke"), Aritzek aldatu du. Berrikuspen-zikloa berriz. Onartuta. Merge egin. Bug-a konponduta produkzioan. Aritzek lehen aldiz benetako industri-fluxua bizi izan du.

💡 **Idea nagusia** Pull Request bat ez da burokrasia: kalitate-kontrola, ezagutzaren transferentzia eta erantzukizun-historia da. Erabaki teknikoetan zer egiten den eta zergatik dokumentatzen da PR-en bidez.

⚠ **Kontuz: API-gakoa Git-era igota** Edozein garatzailek behin egin du: `.env` fitxategi bat Git-era igo, API-gakoarekin. GitHub publikoan, sekretua bot-ek detektatzen dute minutu gutxitan eta erasotzaileek erabiltzen dute. Igo bazaizu, **berehala** giltza aldatu, eta historia garbitu `git filter-branch` edo `BFG Repo-Cleaner`-arekin.

❑ **Hausnartzeko** Inoiz galdu duzu lan baten edukia ordenagailua hondatu delako edo fitxategi bat ustekabez ezabatu duzulako? Git-ekin eta GitHub-ekin (urruneko biltegia) hori berriz ez gertatuko zaizu — lan guztia hodeian dago kopia bat bezala. Zer suposatzen du horrek zure eguneroko lanaren segurtasunerako?

❑ **Ariketa 4.4 • 20 min • banaka Egoera:** 4.3 ariketako repo-a daukazu.
Eginkizuna:

1. Sortu `.gitignore` fitxategia Python proiektuetarako (eskuz idatziz edo `gitignore.io`-tik kopiatuz).
2. Sortu `.venv/` karpeta eta egiaztatu `.gitignore`-k ondo ezkutatzen duela (`git status` egitean ez zaizu agertu behar).
3. Sortu `.env` fitxategi fikziozko `API_KEY=ezkutua123` edukiarekin. Egiaztatu ezkutatzen dela. 4. Idatzi PR txantilo bat (`PULL_REQUEST_TEMPLATE.md`) zure repoa konfigurazioaren parte gisa.

5. Laburpena eta gakoak

Gai honek modulu osoaren oinarria jarri du: **lengoiak** ezagutu ditugu (Python nagusi, R/Java/JS osagarri), **marko-lengoiak** (XML, HTML, JSON, YAML, bakoitza bere lekuan), **ingurune profesionala** (Antigravity editore gisa, ingurune birtualak proiektuen isolamendurako, pip pakete-kudeaketa, `requirements.txt` erreproduzigarritasuna), **Python aurreratua** (list comprehensions, lambda/map/filter, JSON/YAML/Pickle) eta **Git** (bertsio-kontrola, branch-ak, PRs).

Hauek dira hurrengo gai guztien azpiegitura: 2. gaian datu-zientzia, 3. gaian AA programazioa eta GenAI, 4. gaian konbergentzia teknologikoa, eta 5. gaian agente korporatiboak landuko ditugu. Gai bakoitzean ari zaren lanak gai honetako ingurunean exekutatu dira beti: editore on bat, ingurune birtual aktibo bat, paketeak ondo kudeatuta, eta dena Git-en bertsionatua.

! Ideia nagusia (gaiarena) *Software-ingeniari profesional ona ez da algoritmo konplexuetan adituena bakarrik — bere ingurune eta lan-fluxuak profesionalki kudeatzen dakiena ere bada. Ingurune eskasa duen genio batek ingurune ona duen edozeinek baino kalitate baxuagoko softwarea egiten du.*

☐ **Hausnartzeko (gaiarena)** *Ondorengo asteetan zerbait berria probatzen duzunean (liburutegi bat, framework bat), gogoratu beti: ingurune birtuala lehenik, `requirements.txt` ondoren, Git-en commit txikiak etengabe, eta Antigravity-ren agenteari konfiantzaz galdetu zerbait blokeatzen zaizunean.*

6. Erreferentziak

Dokumentazio ofiziala

- [Python 3 dokumentazioa](https://docs.python.org/3/)
- [Antigravity dokumentazioa](https://antigravity.google)
- [VS Code Python tutoriala](https://code.visualstudio.com/docs/python/python-tutorial)
- [pip dokumentazioa](https://pip.pypa.io/en/stable/)
- [conda dokumentazioa](https://docs.conda.io/en/latest/)
- [Pro Git liburua (doakoa, euskaraz)](https://git-scm.com/book/eu/v2)

Tutorialak eta baliabide osagarriak

- [Real Python — List Comprehensions](https://realpython.com/list-comprehension-python/)
- [Real Python — Lambda Functions](https://realpython.com/python-lambda/)
- [GitHub Learning Lab](https://lab.github.com/)
- [Conventional Commits](https://www.conventionalcommits.org/)

Tresnak

- [gitignore.io](https://www.toptal.com/developers/gitignore) — `.gitignore` sortzailea
- [PyPI](https://pypi.org/) — Python paketeen biltegi nagusia

Lege-erreferentziak

- Errege Dekretu 279/2021 — Adimen Artifizial eta Big Data Espezializazio-ikastaroaren oinarrizko curriculuma.